

AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

408/1, Kuratoli, Khilkhet, Dhaka 1229, Bangladesh



Project Name: Make-up class management system

No	Name	ID	Program	Contribution
1	MD REDWANUL HOQUE	23-50057-1	CSE	25% -> Project Proposal, User Interface design, Schema Diagram, ER-Diagram, Query
2	DIPU ROY	23-50420-1	CSE	25% -> Scenario Description, Normalization, Class Diagram
3	DIPTA MONI SINHA	23-54437-3	CSE	25% -> Use Case Diagram, Conclusion, Query
4	AKHTAB, SHAH MD. AKIL	21-44736-1	CSE	25% -> Activity Diagram, Query

Course Name: Advance database management system

Section: B

Contents:	Page
Introduction	3
Project Proposal	4
Class Diagram	5
Use Case Diagram	6
Activity Diagram	7
User Interface Planning	8-10
Scenario Description	11
ER Diagram	12
Normalization	13-19
Schema Diagram	19
Table Creation Using SQL	20-30
Data Insertion Using SQL	30-41
Basic PL/SQL	41-57
Advance PL/SQL	58-74
Conclusion	74-75

1. Introduction

Sometimes a teacher cannot take a regular class, so an extra class is held later to cover it. This extra class is called a "make-up class." Normally, arranging one means the teacher has to talk to students to find a free time, check for an empty room, and then get permission from the head of the department. Doing all this by hand takes time and often causes confusion.

The Make-up Class Management System is a project designed to make this process simple. It gives students and teachers an easy way to arrange make-up classes without the usual back-and-forth. Teachers can see when students are free and which rooms are open, then request permission from the head of the department, who approves the class only when a suitable time and room are available. This keeps everything organized and easy to track.

2. Project Proposal

2.1 Overview of the Proposed System

We propose to build a Make-up Class Management System for the university that handles the entire process of arranging a make-up class from start to finish. The system will have three types of users: Student, Teacher, and Head of Department (HOD). Each user will log in to their own interface and will only see the information and actions relevant to their role.

2.2 Objectives

The project has the following objectives:

- To let teachers request a make-up class when a regular class is missed.
- To let students report which time slots they are free for a make-up class.
- To automatically show which classrooms are free at a chosen time.
- To route every make-up class request to the Head of Department for approval.
- To keep a proper record of every make-up class: who requested it, who approved it, and where and when it will be held.
- To reduce the manual effort and miscommunication involved in arranging make-up classes.
- To notify students on their mobile or device as soon as a make-up class is arranged.

2.3 Plan of the Project (User Roles and Workflow)

Step 1 — Teacher initiates request: A teacher who has missed a regular class logs in and starts a make-up class request for that course and section.

Step 2 — Student availability: Before sending the request forward, the teacher asks the students of that class when they are free. Students submit the time slots that work for them through their own interface.

Step 3 — Room availability: The system checks and shows which classrooms are empty during the time slots that most students agreed on.

Step 4 — Teacher submits request: Using the matched time and an available room, the teacher sends a formal make-up class request to the Head of Department.

Step 5 — HOD approval: The Head of Department reviews the request. If the chosen room and time do not conflict with anything else, the request is approved; otherwise it is sent back to the teacher for a new time or room.

Step 6 — Confirmation and notification: Once approved, the make-up class is confirmed and a notification is sent straight to each student's mobile or device, showing the course, room, date, and time. The teacher also receives a confirmation. The system keeps a record of the class, including the room, date, time, and approval status, for future reference.

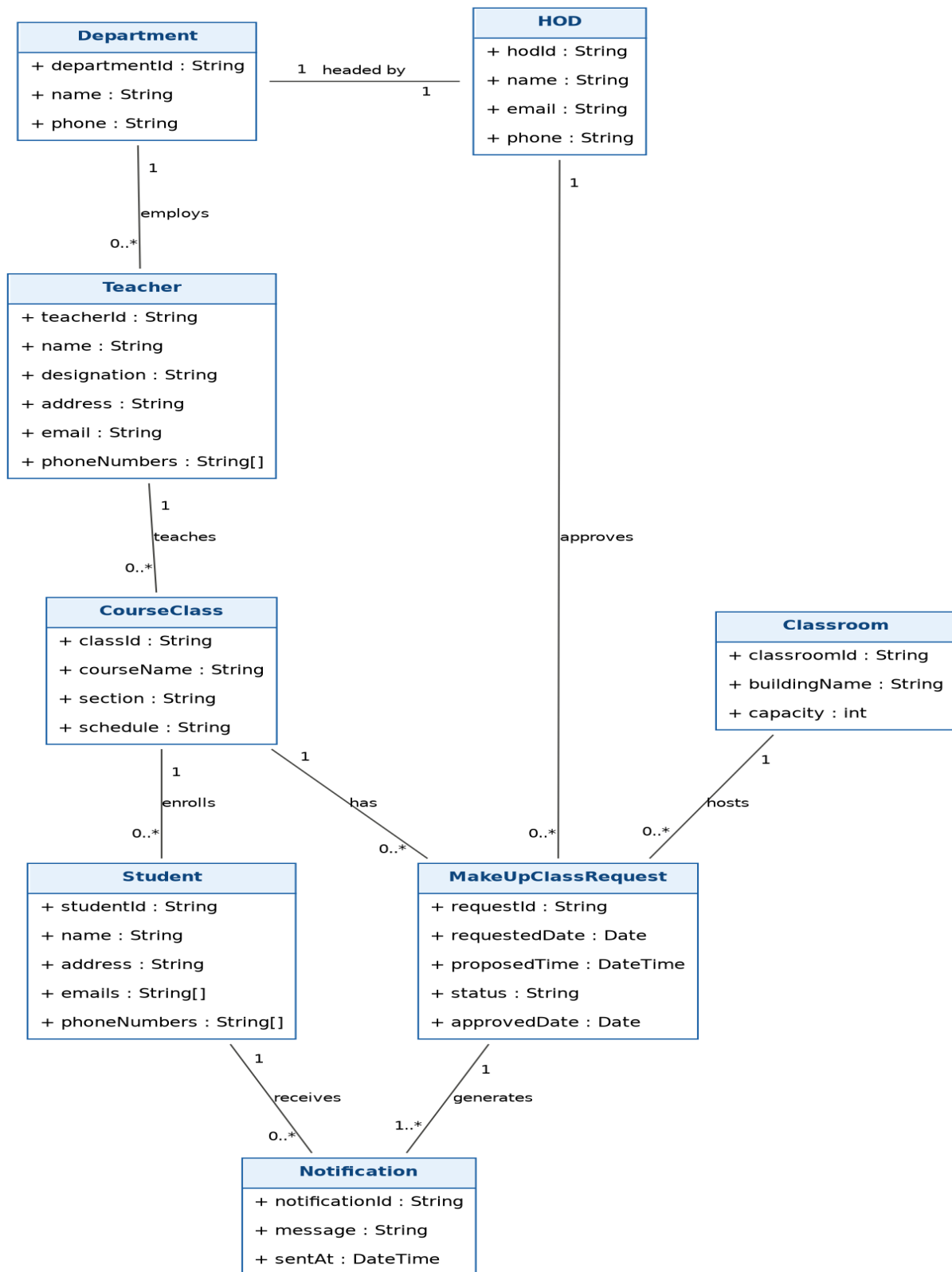
2.4 Scope of the Project

- Separate interfaces for Student, Teacher, and Head of Department.
- Collecting and matching student free time for a make-up class.
- Checking and displaying classroom availability.
- Sending, approving, or rejecting make-up class requests.
- Maintaining a history/record of all make-up classes held.
- Sending a notification to the student's mobile or device once a make-up class is arranged.

2.5 Expected Benefits

With this system, teachers will spend far less time coordinating make-up classes by hand, students will only be asked to pick from realistic time options, and the Head of Department will have a clear, organized way to approve requests without room or time conflicts. Overall, the make-up class process becomes faster, fairer, and easier to track for everyone involved.

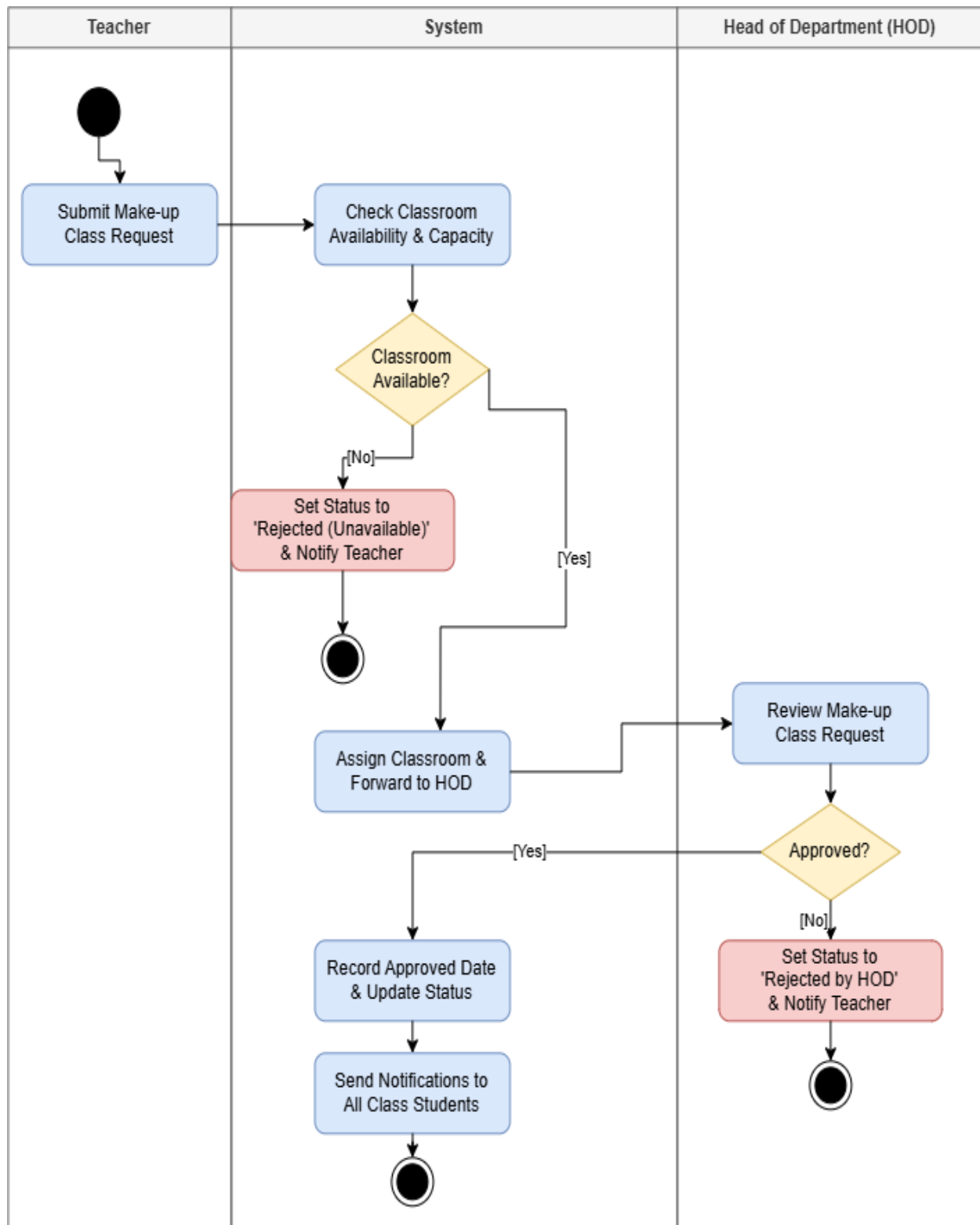
Class Diagram:



Use Case Diagram:



Activity Diagram:



User Interface Planning:

Login page

Login Page

app.makeupclass.edu/login

Make-up Class System

Log In

Login As

Student

ID / Email

e.g. 21-12345-1

Password

[Forgot password?](#)

Log In

Don't have an account?

[Sign Up](#)

Sign-Up Page:

Sign-Up Page

app.makeupclass.edu/signup

Make-up Class System

Create Account

Register As

Teacher

Full Name

e.g. Nusrat Rahman

Identification Number

e.g. T-00231

Email

name@akub.edu

Phone Number

01XXXXXXXXX

Password

Confirm Password

Create Account

Already have an account?

[Log In](#)

Student Page:

Student — My Availability & Notifications

app.makeupclass.edu/student/home

Make-up Class System

Set My Free Time

CSE221 — Section B requested a make-up class. Select the slots that work for you:

Mon 2–3pm

Tue 10–11am

Wed 1–2pm

Thu 3–4pm

Fri 11–12pm

Sat 9–10am

Submit Availability

Notifications

Make-up class CONFIRMED — CSE221, Room 402, Wed 1–2pm

Teacher requested your free time for CSE221

Upcoming Make-up Classes

CSE221 — Data Structures

Wed, Room 402, 1:00–2:00 PM

Teacher Page:

Teacher — Request a Make-up Class

app.makeupclass.edu/teacher/request

Make-up Class System

Request Make-up Class

Class / Section

CSE221 — Section B

Student Free Time Slots

Mon 2–3pm

Tue 10–11am

Wed 1–2pm

Thu 3–4pm

Fri 11–12pm

Sat 9–10am

Available Classrooms (Wed 1–2pm)

Room 402 ☐ available

Room 305 ☐ available

Lab 2 ☒ booked

Send Request to Head of Department

My Requests

Wed 1–2pm · Room 402 — Pending

Mon 9–10am · Room 305 — Approved

HOD Page:

Head of Department — Approve Requests

app.makeupclass.edu/hod/requests

Make-up Class System

Pending Make-up Class Requests

N. Rahman — CSE221
Room 402 · Wed, 24 Jul · 1:00–2:00 PM

Approve Reject

A. Karim — MAT101
Room 305 · Thu, 25 Jul · 4:00–5:00 PM

Approve Reject

S. Chowdhury — ENG102
Room Lab 1 · Fri, 26 Jul · 10:00–11:00 AM

Approve Reject

Room Availability Check

Selected room & time

☐ Room 402 is free on Wed, 24 Jul, 1:00–2:00 PM

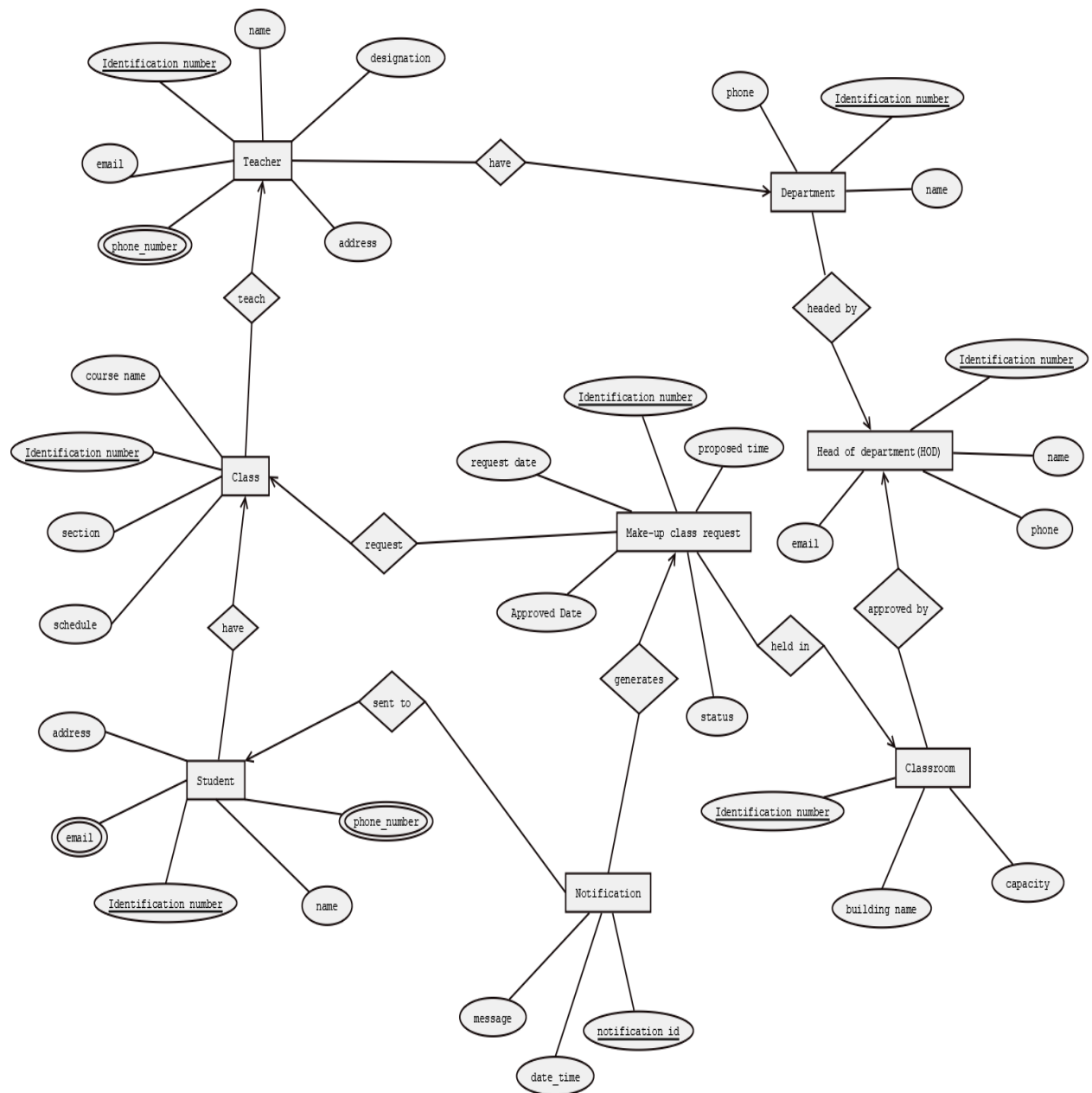
Recently Approved

M. Islam · CSE315 · Room 210 · Wed, 22 Jul · Approved

Scenario Description:

Scenario: In a Make-up Class Management System, one Department may have many Teachers. But one Teacher works in one Department only. Each Teacher has a unique identification number, name, designation, address, email and phone number. A Teacher can have more than one phone number. Department has a unique identification number, name and phone. One Department is headed by exactly one Head of Department (HOD). One HOD heads exactly one Department. HOD is defined by a unique identification number, name, email and phone. One Teacher may teach many Classes, but one Class is taught by exactly one Teacher. Each Class has a unique identification number, course name, section, and regular schedule. A Class may have many Students, but one Student belongs to one Class only. Each Student has a unique identification number, name, address, email and phone number. Each Student can have more than one email address and phone number. One Class may have many Make-up Class Requests, but one Make-up Class Request belongs to exactly one Class. Each Make-up Class Request has a unique identification number, requested date, proposed time and status. One Make-up Class Request must be held in exactly one Classroom, but one Classroom may be used for many Make-up Class Requests. Classroom has a unique identification number, building name and capacity. One Make-up Class Request must be approved by exactly one HOD, but one HOD may approve many Make-up Class Requests. Record approved date. Once a Make-up Class Request is approved, the system sends many Notifications. one to each Student of that Class, but each Notification belongs to exactly one Make-up Class Request. Notification is sent to exactly one Student. Notification has a unique identification number, message and the date and time it was sent.

ER-Diagram:



Normalization:

Step 1 — Entities, attributes, and relationships (from the scenario)

Entity	Attributes	Primary key
Department	dept_id, dept_name, dept_phone	dept_id
HOD	hod_id, hod_name, hod_email, hod_phone	hod_id
Teacher	t_id, t_name, designation, t_address, t_email, t_phone (multi-valued)	t_id
Class	class_id, course_name, section, schedule	class_id
Student	s_id, s_name, s_address, s_email (multi-valued), s_phone (multi-valued)	s_id
Classroom	room_id, building_name, capacity	room_id
Make-up Class Request	req_id, requested_date, proposed_time, status, approved_date	req_id
Notification	notif_id, message, sent_datetime	notif_id

Relationships / cardinalities:

Department (1) — (N) Teacher

Department (1) — (1) HOD (mutual one-to-one: one dept has exactly one HOD, one HOD heads exactly one dept)

Teacher (1) — (N) Class

Class (1) — (N) Student

Class (1) — (N) Make-up Class Request

Classroom (1) — (N) Make-up Class Request

HOD (1) — (N) Make-up Class Request (approves)

Make-up Class Request (1) — (N) Notification

Student (1) — (N) Notification

Relation 1 — STAFF (Department + HOD + Teacher)

UNF

Staff (dept_id, dept_name, dept_phone, hod_id, hod_name, hod_email, hod_phone, t_id, t_name, designation, t_address, t_email, t_phone)

1NF

1. dept_id, dept_name, dept_phone, hod_id, hod_name, hod_email, hod_phone, t_id, t_name, designation, t_address, t_email, t_phone
2. t_id, t_phone

Now every attribute is single-valued $\rightarrow$ relation is in 1NF, key = (t_id, t_phone).

2NF

Must be in 1NF, and must remove partial dependencies (attributes depending on only part of the composite key).

Functional dependencies:

- t_id $\rightarrow$ t_name, designation, t_address, t_email, dept_id (depends on t_id only — partial)
- dept_id $\rightarrow$ dept_name, dept_phone, hod_id (depends on t_id only, transitively — partial)
- hod_id $\rightarrow$ hod_name, hod_email, hod_phone (depends on t_id only, transitively — partial)
- (t_id, t_phone) $\rightarrow$ nothing new — t_phone has no non-key dependents

Remove the partial dependencies $\rightarrow$ split into:

1. t_id, t_name, designation, t_address, t_email, dept_id, dept_name, dept_phone, hod_id, hod_name, hod_email, hod_phone
2. t_id, t_phone

3NF

Must be in 2NF, and must remove transitive dependencies (non-key attribute depending on another non-key attribute).

In table 1: t_id $\rightarrow$ dept_id $\rightarrow$ dept_name, dept_phone, hod_id and dept_id $\rightarrow$ hod_id $\rightarrow$ hod_name, hod_email, hod_phone are both transitive (dept info and HOD info depend on t_id only through dept_id, not directly).

Remove them:

1. t_id, t_name, designation, t_address, t_email, dept_id (Teacher)
2. t_id, t_phone (TeacherPhone)
3. dept_id, dept_name, dept_phone, hod_id (Department)
4. hod_id, hod_name, hod_email, hod_phone (HOD)

Table Creation

1. Teacher (t_id PK, t_name, designation, t_address, t_email, dept_id FK)
2. TeacherPhone (t_id FK, t_phone) PK(t_id, t_phone)
3. Department (dept_id PK, dept_name, dept_phone, hod_id FK, UNIQUE)
4. HOD (hod_id PK, hod_name, hod_email, hod_phone)

Relation 2 — CLASS ROSTER (Teacher + Class + Student)

UNF

classRoster(t_id, t_name, class_id, course_name, section, schedule, s_id, s_name, s_address, s_email, s_phone)

One flat row per student: it repeats their class's info and that class's teacher info.

1NF

s_email and s_phone are both independently multi-valued (a student can have several of each). Move each out to its own line/table so every attribute becomes atomic:

1. t_id, t_name, class_id, course_name, section, schedule, s_id, s_name, s_address
2. s_id, s_email
3. s_id, s_phone

Relation 1 now has a single-attribute key, s_id (one student = one row; a student belongs to exactly one class).

2NF

The key of relation 1 is the single attribute s_id. A single-attribute key cannot have a partial dependency, so relation 1 is already in 2NF. (Relations 2 and 3 are also trivially in 2NF — each has a two-attribute key but no non-key attributes at all.)

3NF

Functional dependencies in relation 1:

$s_id \rightarrow s_name, s_address, class_id$

$class_id \rightarrow course_name, section, schedule, t_id$ (transitive: depends on s_id only through $class_id$)

$t_id \rightarrow t_name$ (transitive: depends on s_id only through $class_id \rightarrow t_id$)

Remove the transitive dependencies:

1. $s_id, s_name, s_address, class_id$ (Student)
2. $class_id, course_name, section, schedule, t_id$ (Class)
3. ~~t_id, t_name~~ (already captured in Teacher, Relation 1)
4. s_id, s_email (StudentEmail)
5. s_id, s_phone (StudentPhone)

Table Creation

1. Student (s_id PK, s_name , $s_address$, $class_id$ FK)
 2. Class ($class_id$ PK, $course_name$, $section$, $schedule$, t_id FK)
 3. StudentEmail (s_id FK, s_email) PK(s_id, s_email)
 4. StudentPhone (s_id FK, s_phone) PK(s_id, s_phone)
- (Teacher table already exists from Relation 4 — no duplicate table needed, just the t_id FK link.)

Relation 3 — MAKE-UP REQUEST (Class + Classroom + HOD + Request + Notification + Student)

UNF

The lowest-grain fact here is one Notification (one row is sent per student, per approved request), so a flat "notification log" row would repeat everything above it:

requestLog(notif_id, message, sent_datetime, req_id, requested_date, proposed_time, status, approved_date, class_id, course_name, room_id, building_name, capacity, hod_id, hod_name, s_id, s_name)

1NF

No attribute above is multi-valued (each notification is one row, one message, one timestamp, one student) — the relation is already in 1NF, key = notif_id.

1. notif_id, message, sent_datetime, req_id, requested_date, proposed_time, status, approved_date, class_id, course_name, room_id, building_name, capacity, hod_id, hod_name, s_id, s_name

2NF

Key is the single attribute notif_id → no composite key, so no partial dependency is possible. Already in 2NF.

3NF

Functional dependencies:

notif_id → message, sent_datetime, req_id, s_id

req_id → requested_date, proposed_time, status, approved_date, class_id, room_id, hod_id (transitive)

class_id → course_name (transitive, one level deeper)

room_id → building_name, capacity (transitive)

hod_id → hod_name (transitive)

s_id → s_name (transitive)

Remove every transitive dependency:

1. notif_id, message, sent_datetime, req_id, s_id (Notification)
2. req_id, requested_date, proposed_time, status, approved_date, class_id, room_id, hod_id (MakeupClassRequest)
3. ~~class_id, course_name~~ (already captured — Class, Relation 2)
4. room_id, building_name, capacity (Classroom)
5. ~~hod_id, hod_name~~ (already captured — HOD, Relation 1)

6. ~~s_id, s_name~~ (already captured — Student, Relation 2)

Table Creation

1. Notification(notif_id PK, message, sent_datetime, req_id FK, s_id FK)
2. MakeupClassRequest(req_id PK, requested_date, proposed_time, status, approved_date, class_id FK, room_id FK, hod_id FK)
3. Classroom(room_id PK, building_name, capacity)

Final consolidated 3NF schema (all three forms merged, duplicates removed)

1. Department(dept_id PK, dept_name, dept_phone, hod_id FK UNIQUE)
2. HOD(hod_id PK, hod_name, hod_email, hod_phone)
3. Teacher(t_id PK, t_name, designation, t_address, t_email, dept_id FK)
4. TeacherPhone(t_id FK, t_phone) PK(t_id, t_phone)
5. Class(class_id PK, course_name, section, schedule, t_id FK)
6. Student(s_id PK, s_name, s_address, class_id FK)
7. StudentEmail(s_id FK, s_email) PK(s_id, s_email)
8. StudentPhone(s_id FK, s_phone) PK(s_id, s_phone)
9. Classroom(room_id PK, building_name, capacity)
10. MakeupClassRequest(req_id PK, requested_date, proposed_time, status, approved_date, class_id FK, room_id FK, hod_id FK)
11. Notification(notif_id PK, message, sent_datetime, req_id FK, s_id FK)

Each relationship is correctly enforced in 3NF:

Dept–Teacher (1:N) → dept_id FK sits in Teacher (the "many" side).

Dept–HOD (1:1) → hod_id FK sits in Department with a UNIQUE constraint, so no department can point to two HODs and no HOD can be reused by two departments.

Teacher–Class (1:N) → t_id FK sits in Class.

Class–Student (1:N) → class_id FK sits in Student.

Class–MakeupClassRequest (1:N) and Classroom–MakeupClassRequest (1:N) and HOD–MakeupClassRequest (1:N) → class_id, room_id, hod_id FKs all sit in MakeupClassRequest (the "many" side of all three).

MakeupClassRequest–Notification (1:N) and Student–Notification (1:N) → req_id and s_id FKs both sit in Notification.

Teacher's multiple phone numbers and Student's multiple emails/phones → each pulled into its own weak table keyed on (owner id, value), which is exactly how 1NF eliminates repeating groups. No transitive or partial dependencies remain in any table — every non-key attribute depends on the whole primary key and nothing but the key, so the schema is fully in 3NF.

Schema diagram:

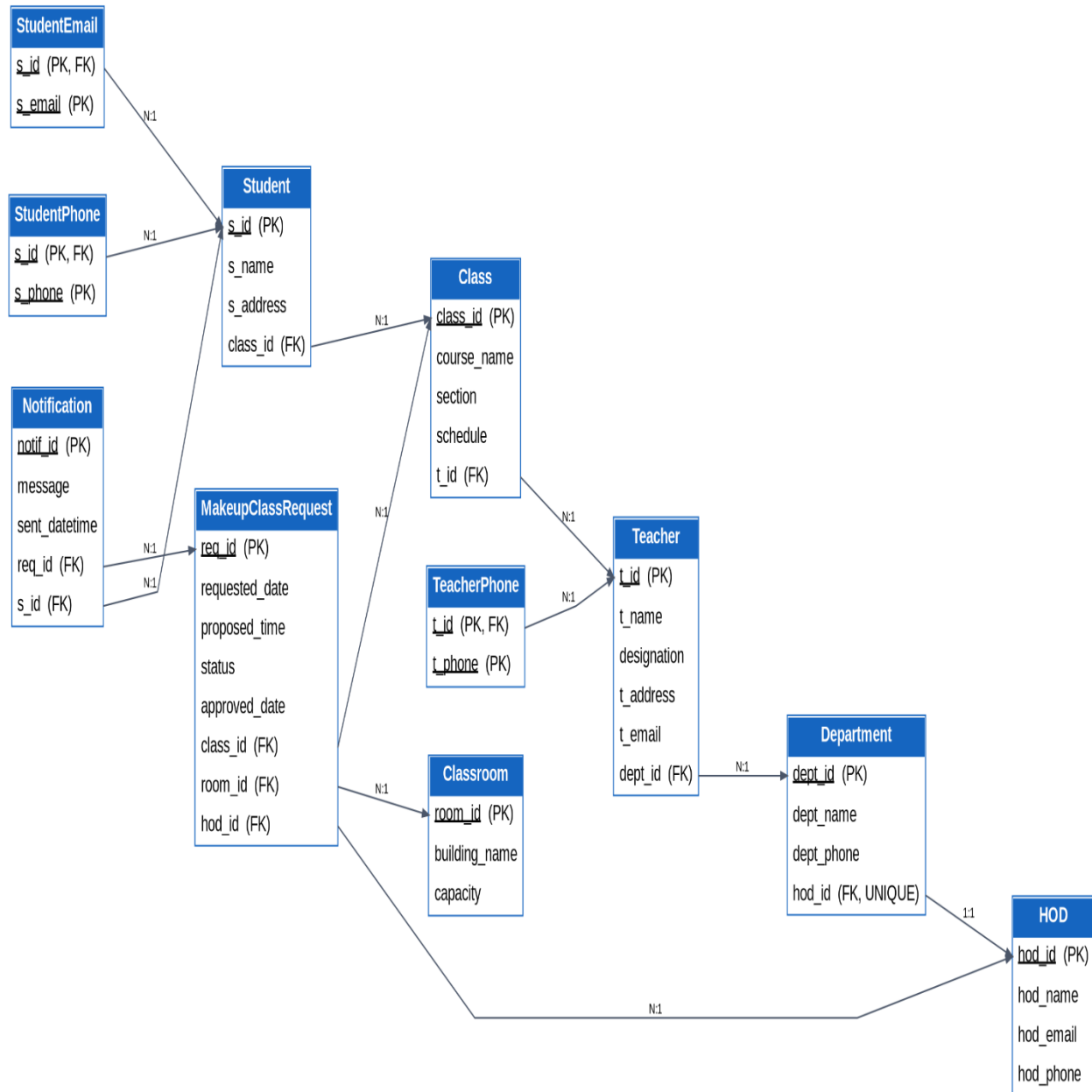
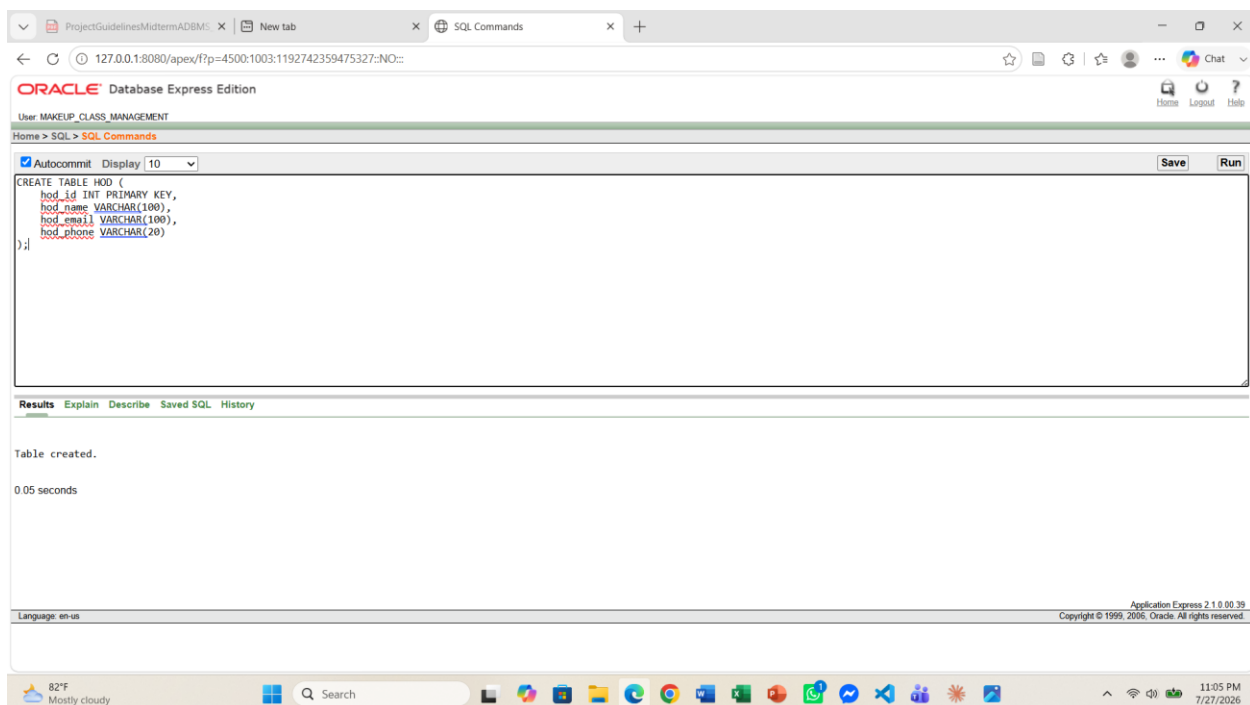
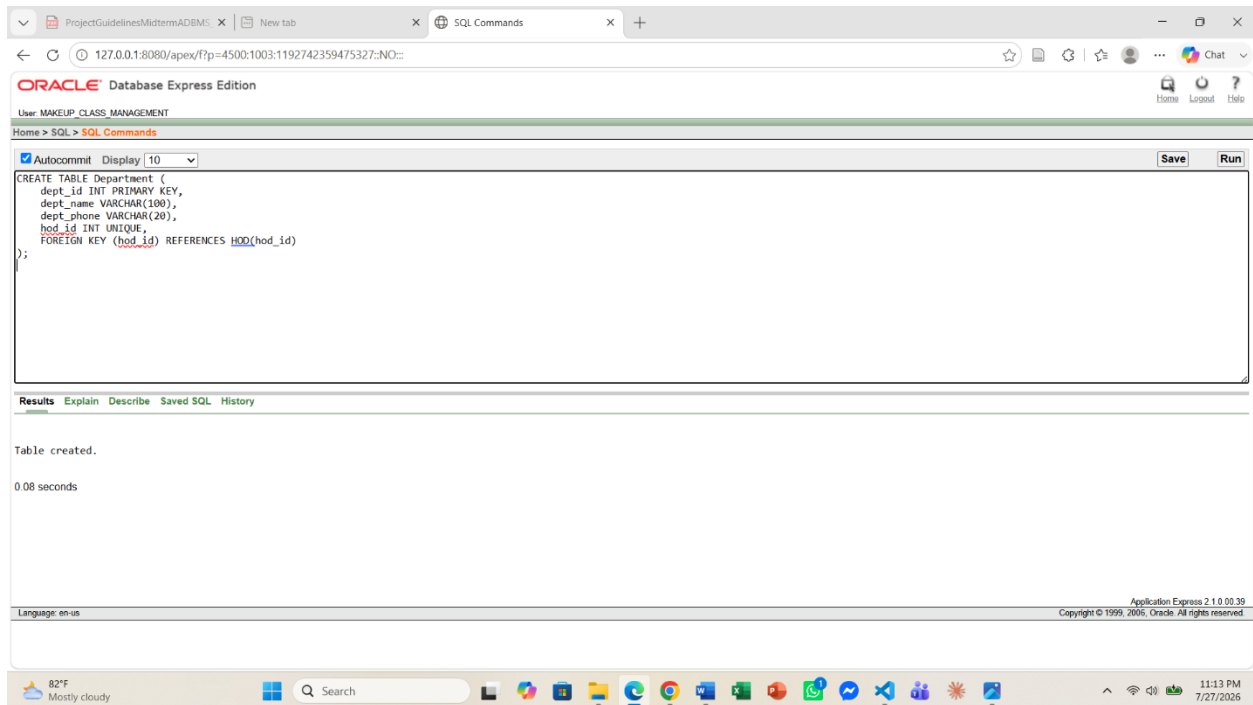


Table creation using SQL:

```
1. CREATE TABLE HOD (  
  hod_id INT PRIMARY KEY,  
  hod_name VARCHAR(100),  
  hod_email VARCHAR(100),  
  hod_phone VARCHAR(20)  
);
```

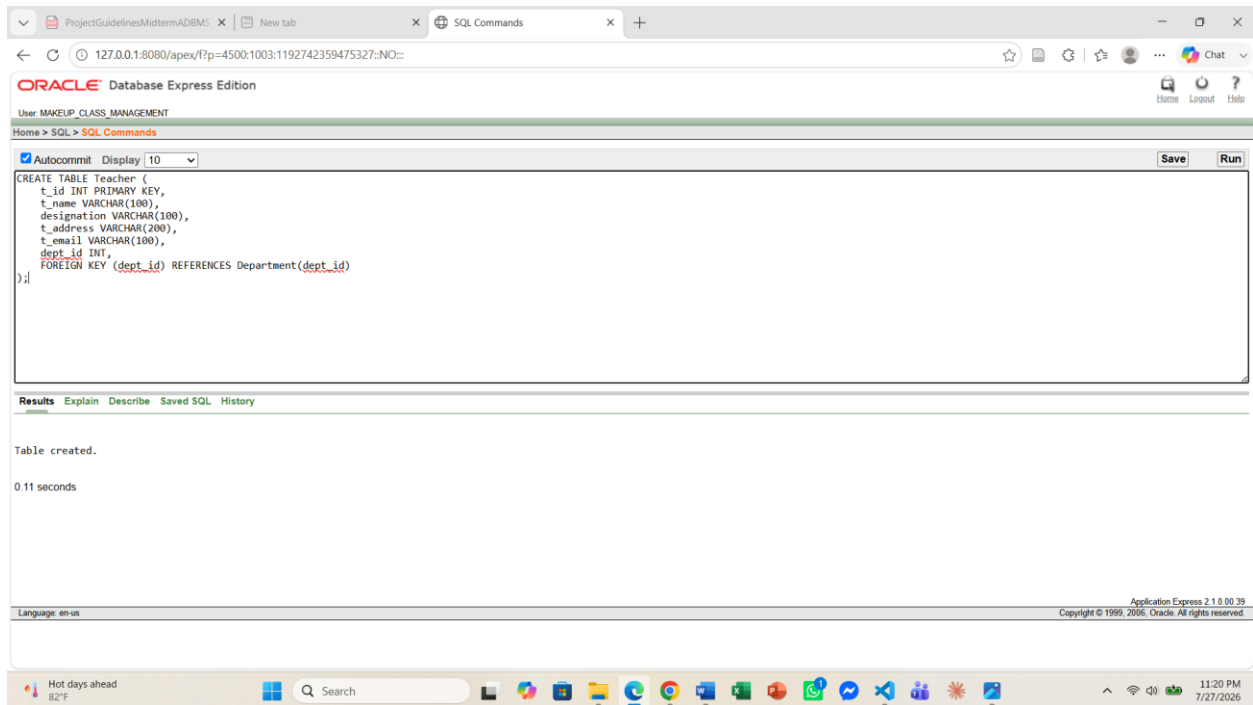


```
2.  
CREATE TABLE Department (  
  dept_id INT PRIMARY KEY,  
  dept_name VARCHAR(100),  
  dept_phone VARCHAR(20),  
  hod_id INT UNIQUE,  
  FOREIGN KEY (hod_id) REFERENCES HOD(hod_id)  
);
```



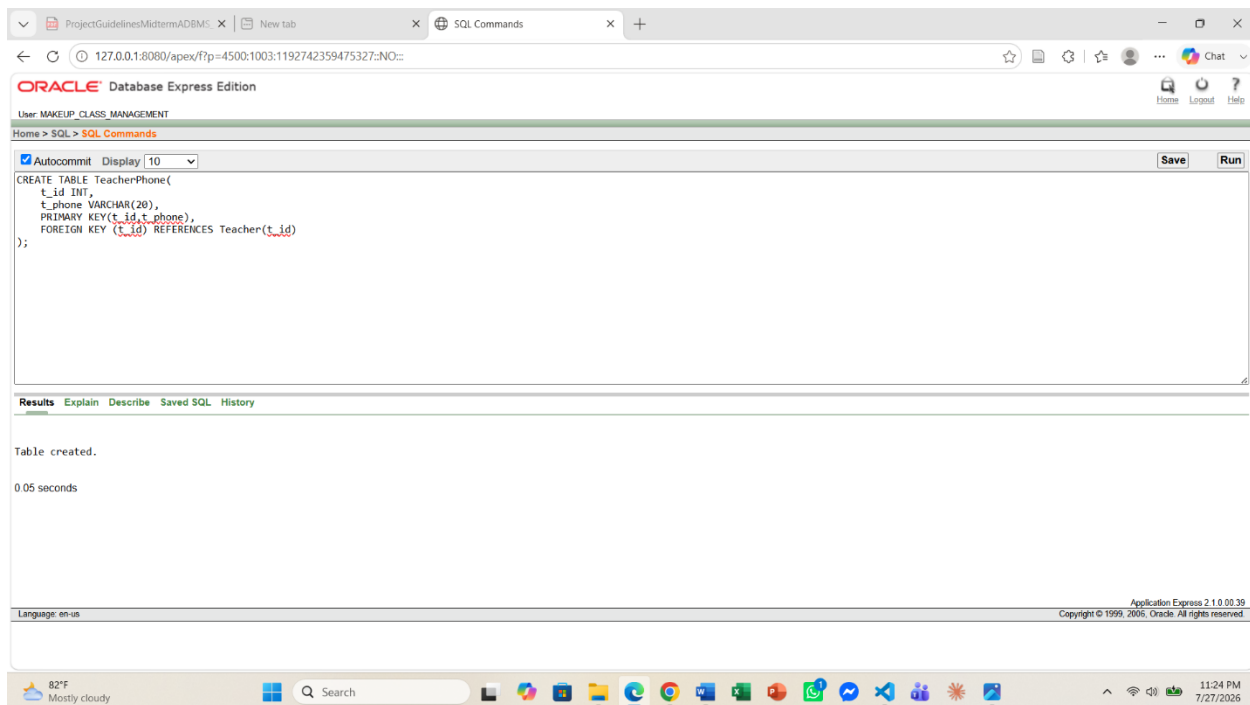
3.

```
CREATE TABLE Teacher (  
  t_id INT PRIMARY KEY,  
  t_name VARCHAR(100),  
  designation VARCHAR(100),  
  t_address VARCHAR(200),  
  t_email VARCHAR(100),  
  dept_id INT,  
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);
```



4.

```
CREATE TABLE TeacherPhone(  
  t_id INT,  
  t_phone VARCHAR(20),  
  PRIMARY KEY(t_id,t_phone),  
  FOREIGN KEY (t_id) REFERENCES Teacher(t_id)  
);
```



5.

CREATE TABLE Class(

class_id INT PRIMARY KEY,

course_name VARCHAR(100),

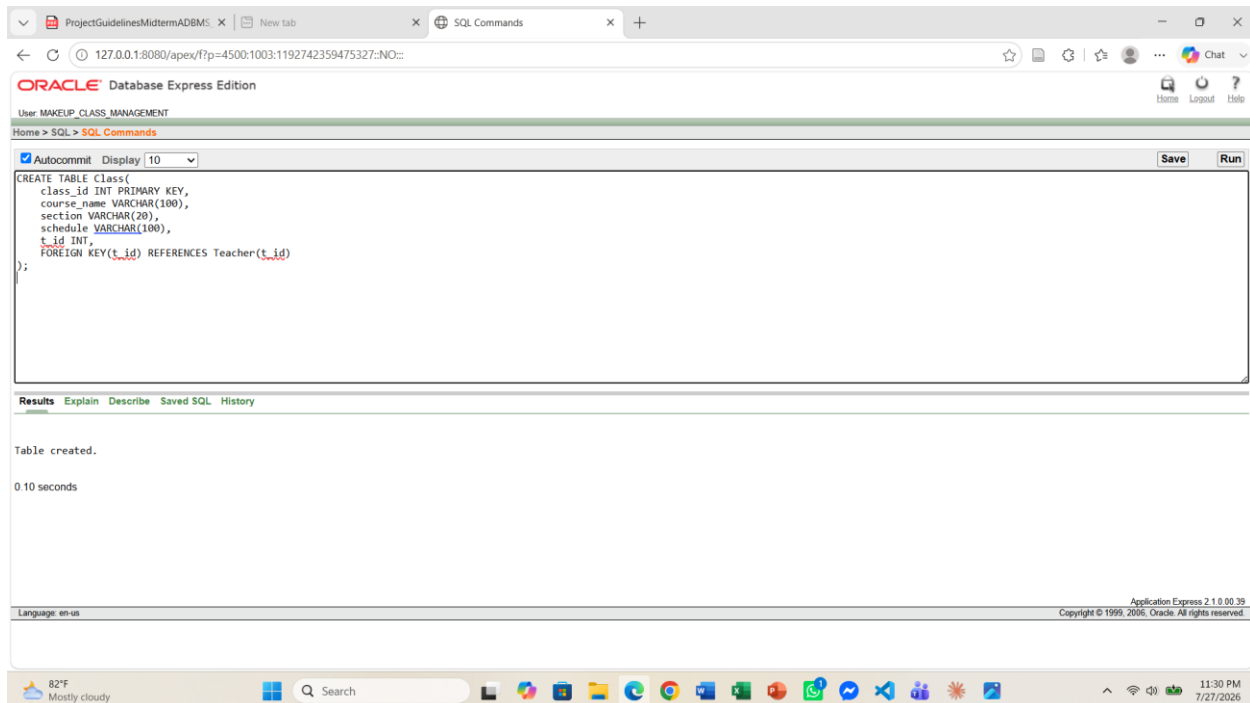
section VARCHAR(20),

schedule VARCHAR(100),

t_id INT,

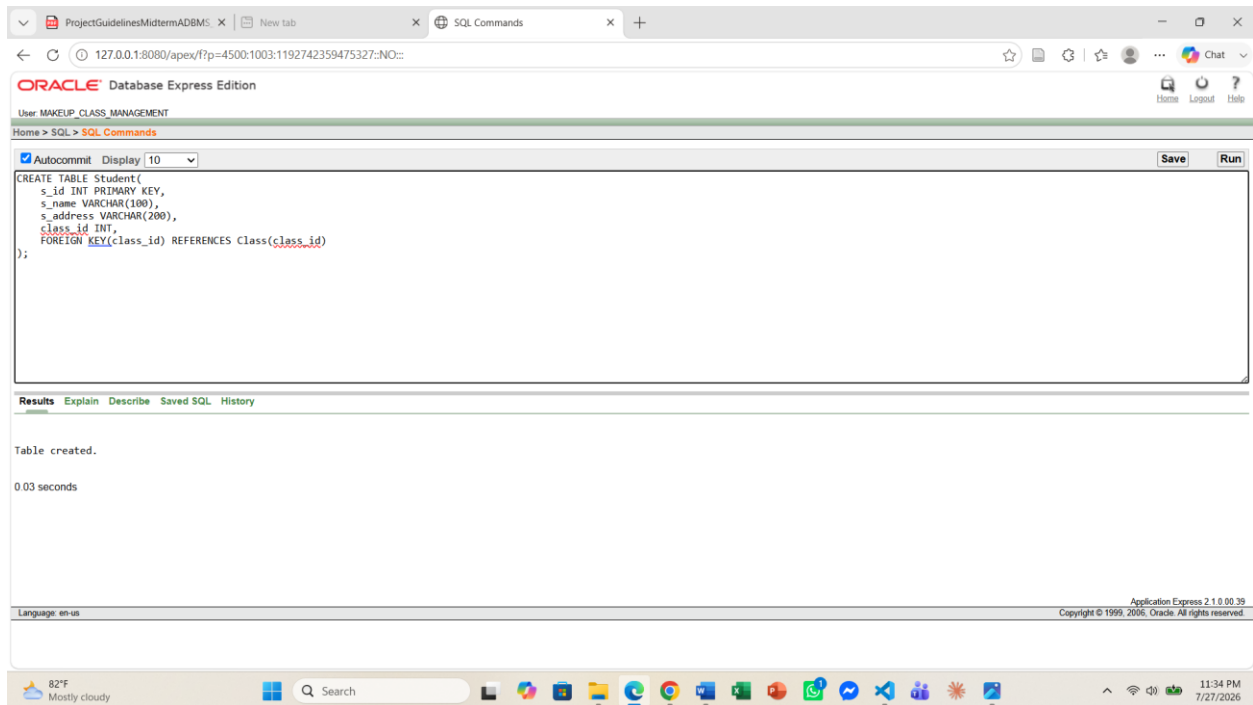
FOREIGN KEY(t_id) REFERENCES Teacher(t_id)

);



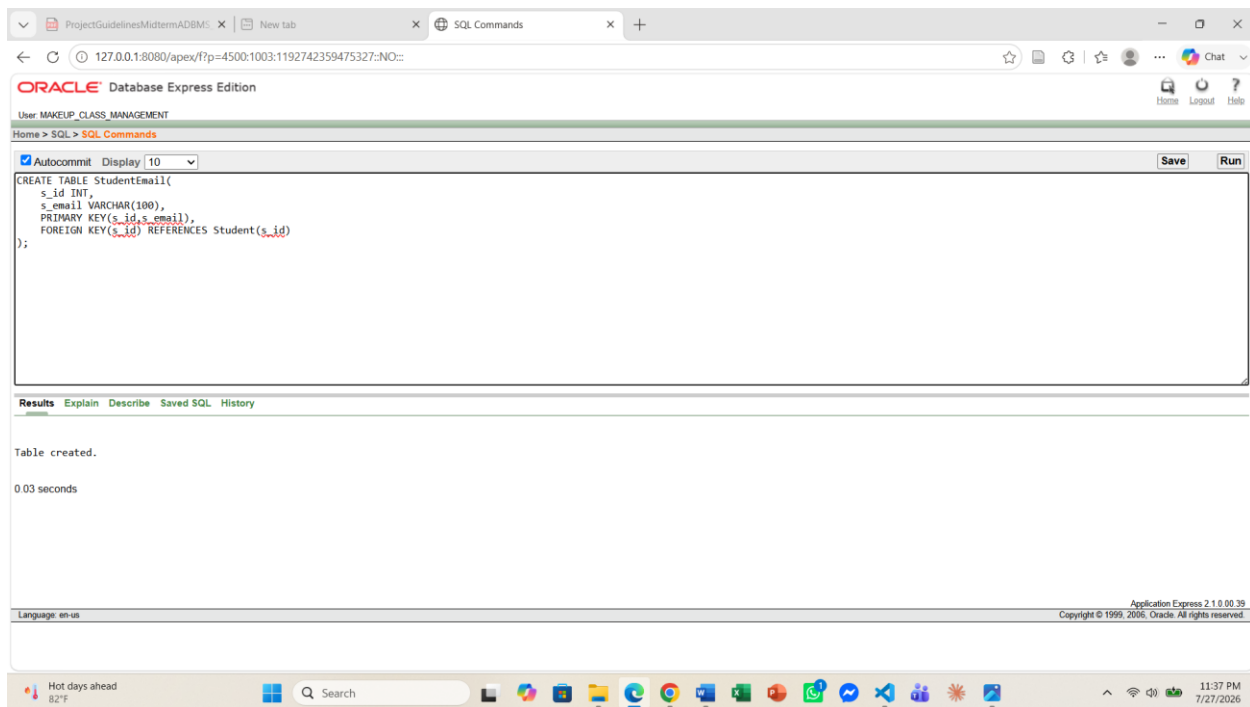
6.

```
CREATE TABLE Student(  
  s_id INT PRIMARY KEY,  
  s_name VARCHAR(100),  
  s_address VARCHAR(200),  
  class_id INT,  
  FOREIGN KEY(class_id) REFERENCES Class(class_id)  
);
```



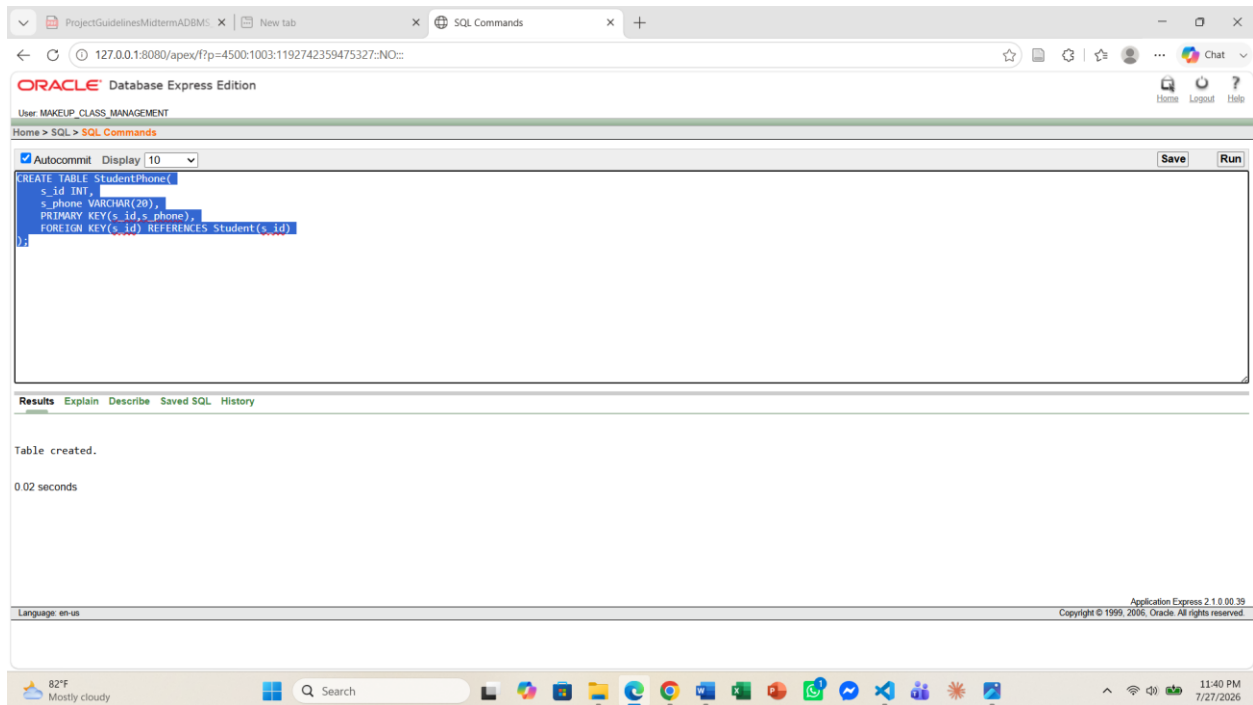
7.

```
CREATE TABLE StudentEmail(  
  
  s_id INT,  
  
  s_email VARCHAR(100),  
  
  PRIMARY KEY(s_id,s_email),  
  
  FOREIGN KEY(s_id) REFERENCES Student(s_id)  
  
);
```



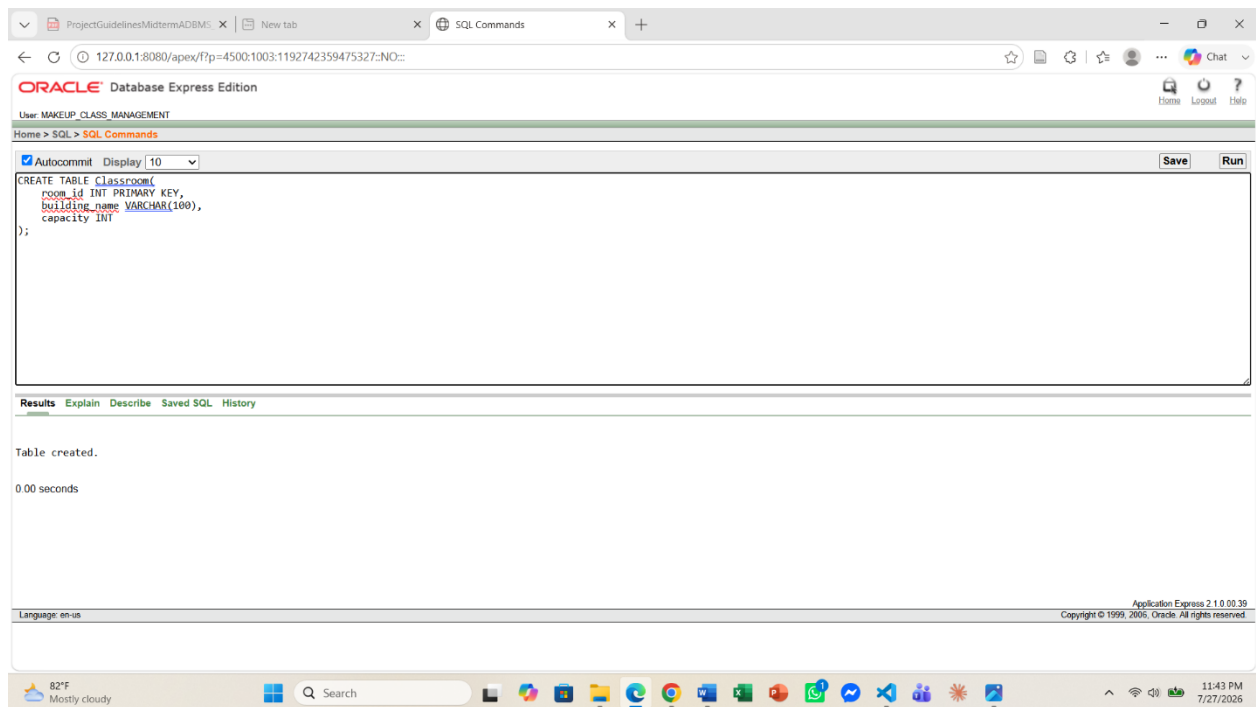
8.

```
CREATE TABLE StudentPhone(  
  s_id INT,  
  s_phone VARCHAR(20),  
  PRIMARY KEY(s_id,s_phone),  
  FOREIGN KEY(s_id) REFERENCES Student(s_id)  
);
```



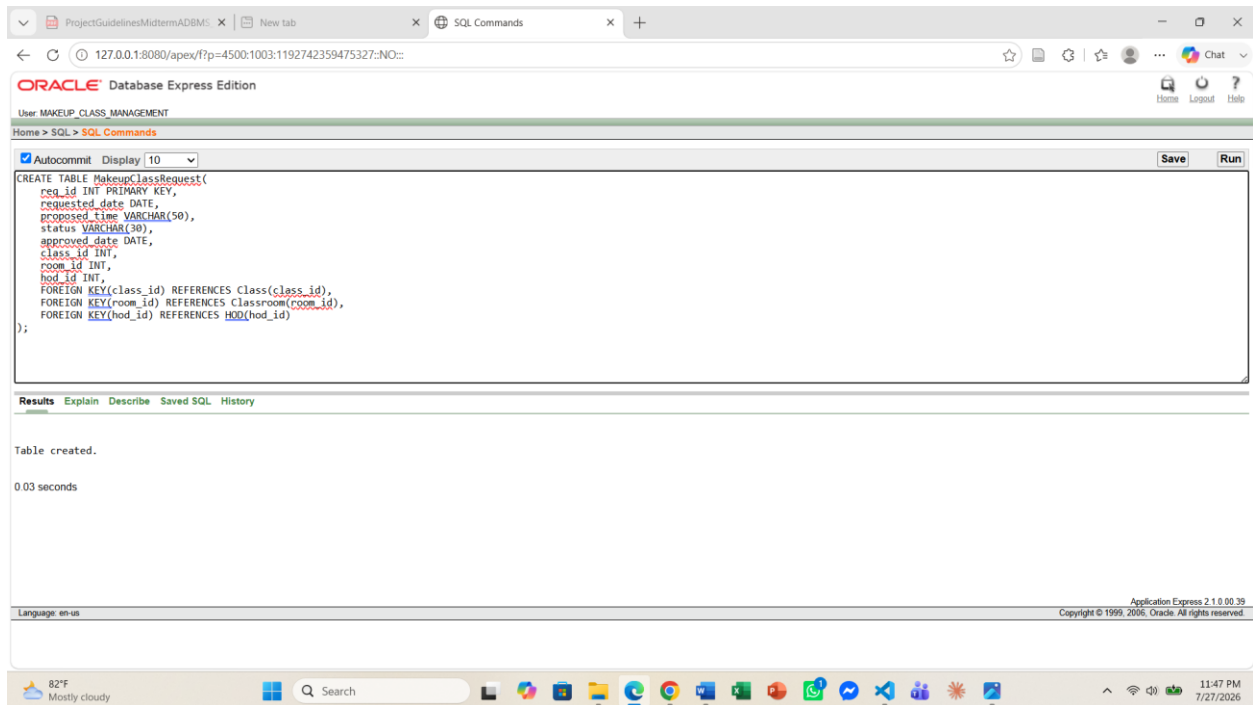
9.

```
CREATE TABLE Classroom(  
  room_id INT PRIMARY KEY,  
  building_name VARCHAR(100),  
  capacity INT  
);
```



10.

```
CREATE TABLE MakeupClassRequest(  
  req_id INT PRIMARY KEY,  
  requested_date DATE,  
  proposed_time VARCHAR(50),  
  status VARCHAR(30),  
  approved_date DATE,  
  class_id INT,  
  room_id INT,  
  hod_id INT,  
  FOREIGN KEY(class_id) REFERENCES Class(class_id),  
  FOREIGN KEY(room_id) REFERENCES Classroom(room_id),  
  FOREIGN KEY(hod_id) REFERENCES HOD(hod_id)  
);
```

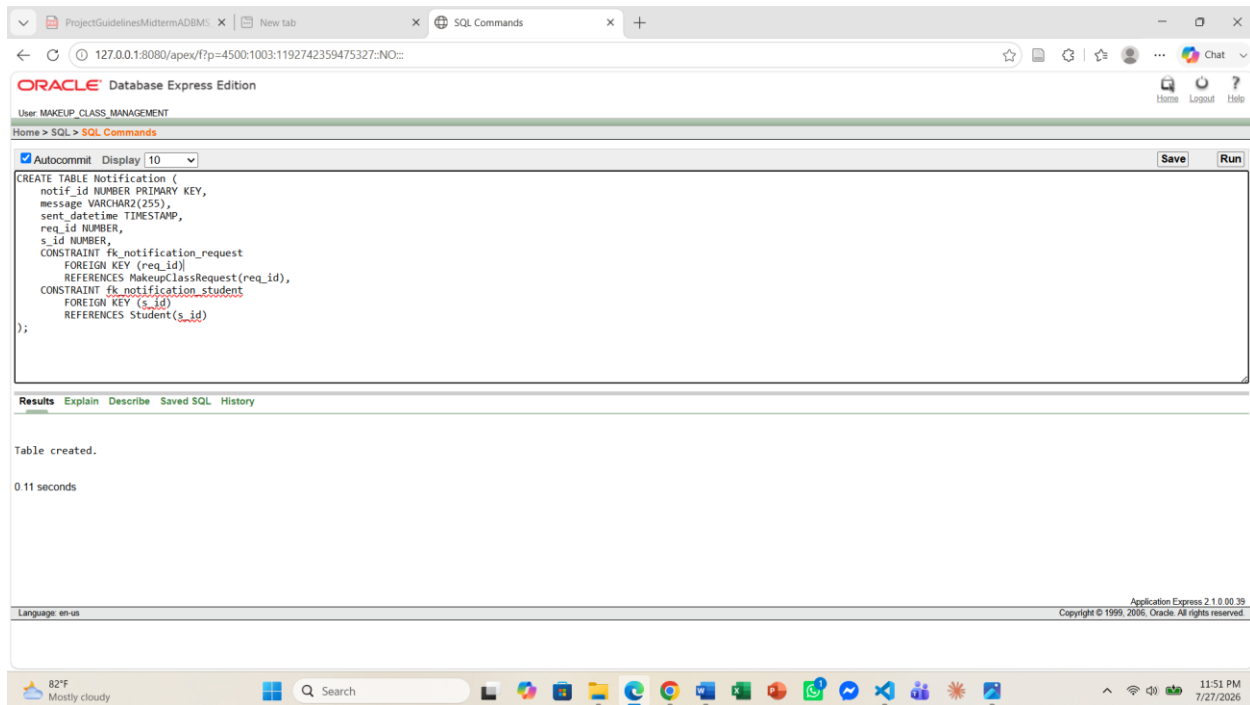


11.

```

CREATE TABLE Notification (
  notif_id NUMBER PRIMARY KEY,
  message VARCHAR2(255),
  sent_datetime TIMESTAMP,
  req_id NUMBER,
  s_id NUMBER,
  CONSTRAINT fk_notification_request
    FOREIGN KEY (req_id)
    REFERENCES MakeupClassRequest(req_id),
  CONSTRAINT fk_notification_student
    FOREIGN KEY (s_id)
    REFERENCES Student(s_id)
);

```



Data Insertion Using SQL:

1.

```
INSERT INTO HOD VALUES (1, 'Dr. Rahman', 'rahman@aiub.edu', '01710000001');
```

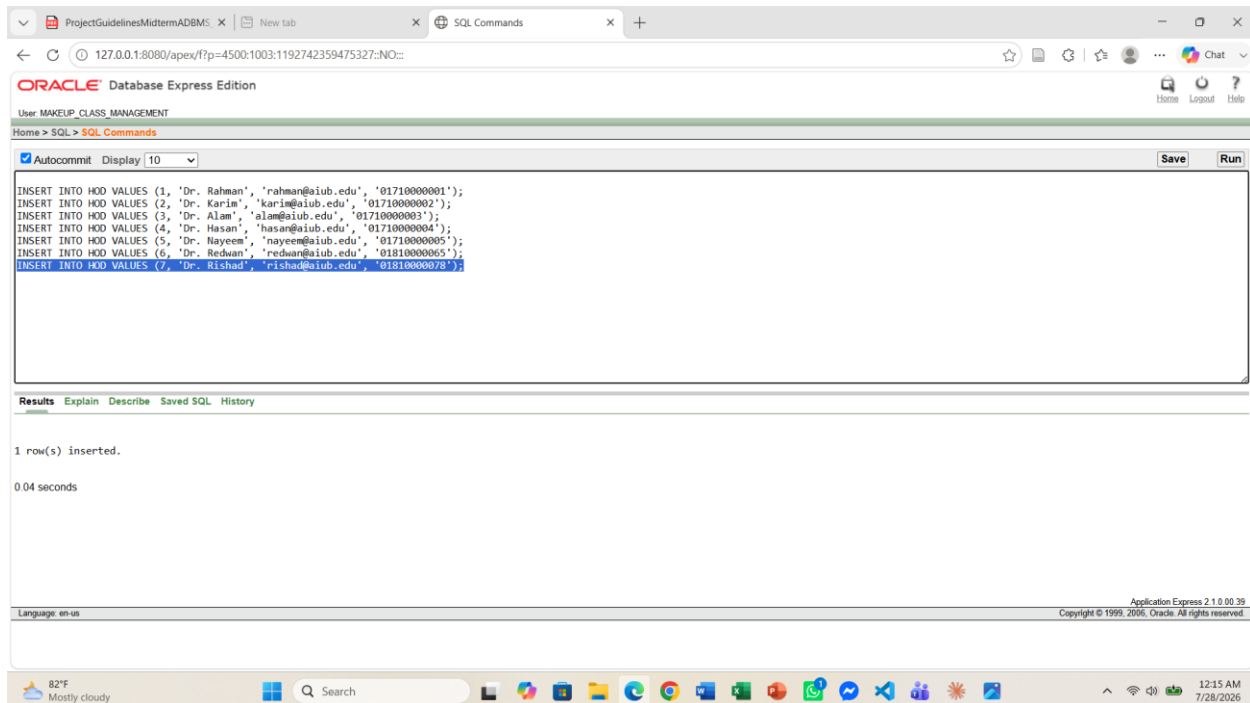
```
INSERT INTO HOD VALUES (2, 'Dr. Karim', 'karim@aiub.edu', '01710000002');
```

```
INSERT INTO HOD VALUES (3, 'Dr. Alam', 'alam@aiub.edu', '01710000003');
```

```
INSERT INTO HOD VALUES (4, 'Dr. Hasan', 'hasan@aiub.edu', '01710000004');
```

```
INSERT INTO HOD VALUES (5, 'Dr. Nayeem', 'nayeem@aiub.edu', '01710000005');
```

```
INSERT INTO HOD VALUES (6, 'Dr. Redwan', 'redwan@aiub.edu', '01810000065');
```



2.

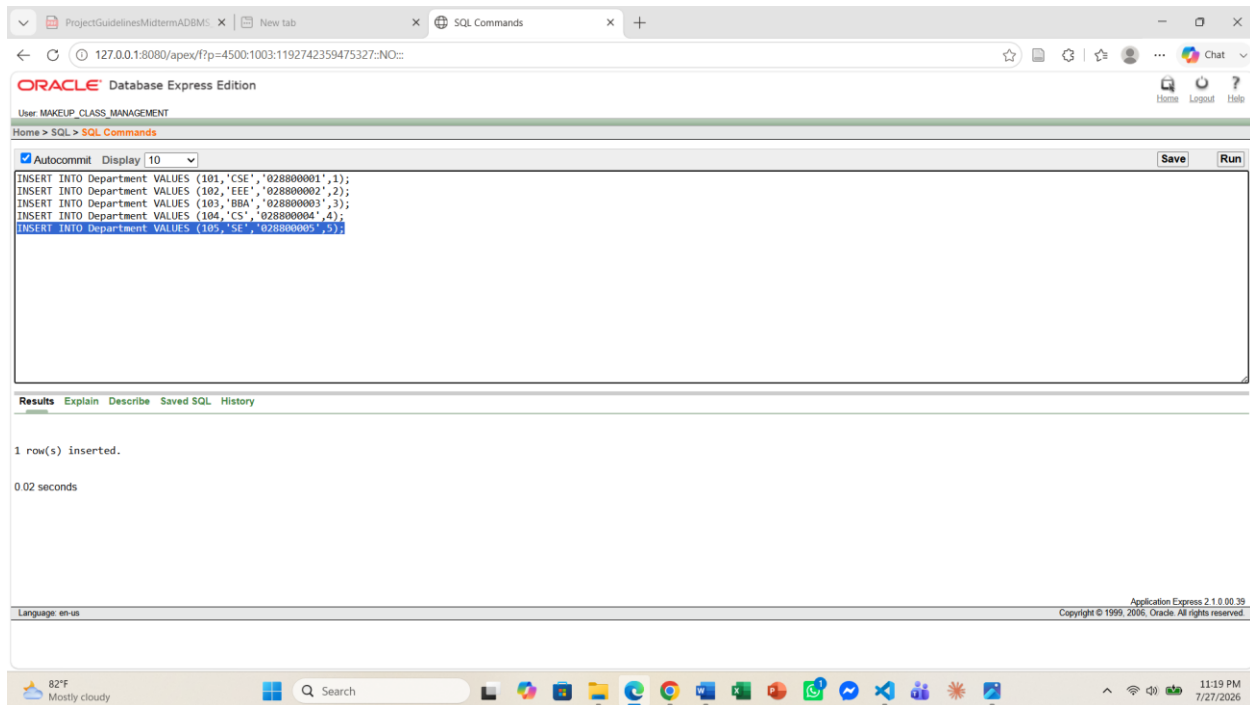
INSERT INTO Department VALUES (101,'CSE','028800001',1);

INSERT INTO Department VALUES (102,'EEE','028800002',2);

INSERT INTO Department VALUES (103,'BBA','028800003',3);

INSERT INTO Department VALUES (104,'CS','028800004',4);

INSERT INTO Department VALUES (105,'SE','028800005',5);



3.

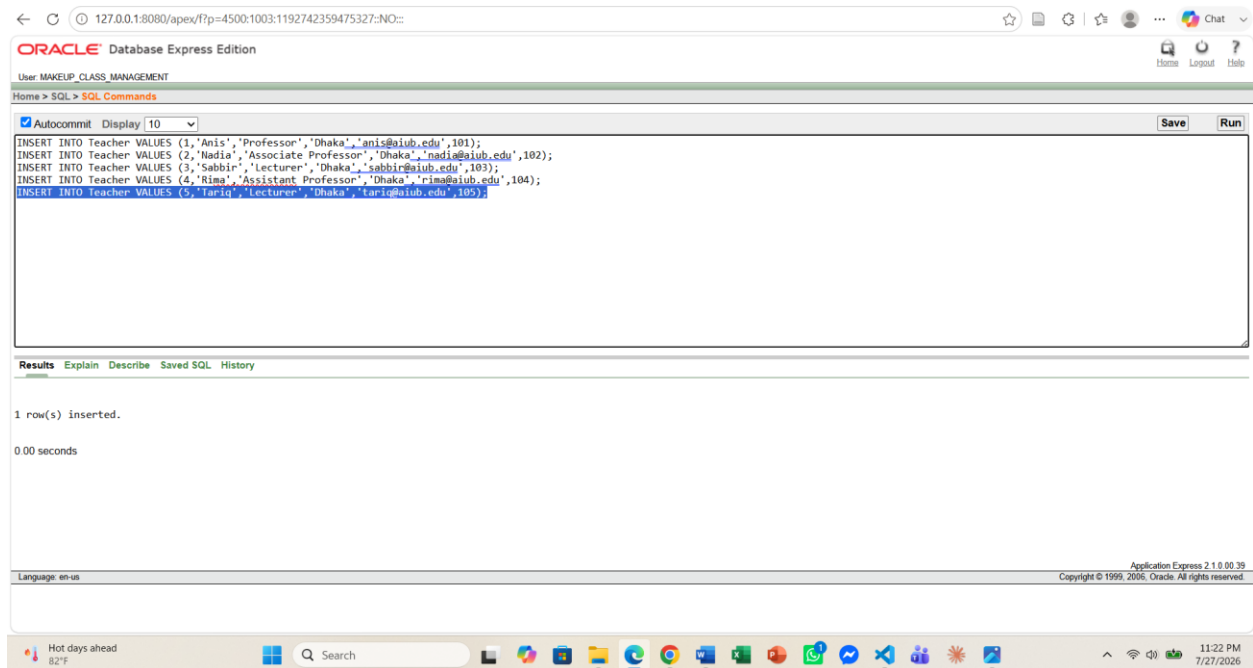
INSERT INTO Teacher VALUES (1,'Anis','Professor','Dhaka','anis@aiub.edu',101);

INSERT INTO Teacher VALUES (2,'Nadia','Associate Professor','Dhaka','nadia@aiub.edu',102);

INSERT INTO Teacher VALUES (3,'Sabbir','Lecturer','Dhaka','sabbir@aiub.edu',103);

INSERT INTO Teacher VALUES (4,'Rima','Assistant Professor','Dhaka','rima@aiub.edu',104);

INSERT INTO Teacher VALUES (5,'Tariq','Lecturer','Dhaka','tariq@aiub.edu',105);



4.

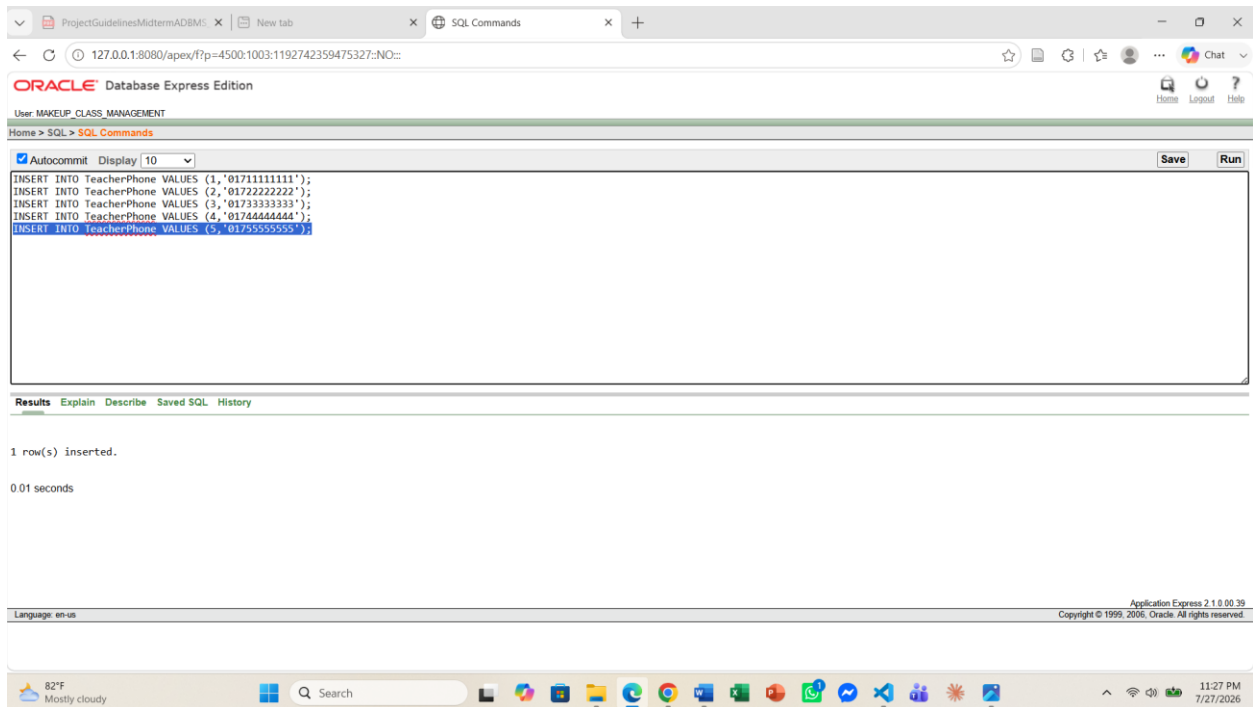
INSERT INTO TeacherPhone VALUES (1,'01711111111');

INSERT INTO TeacherPhone VALUES (2,'01722222222');

INSERT INTO TeacherPhone VALUES (3,'01733333333');

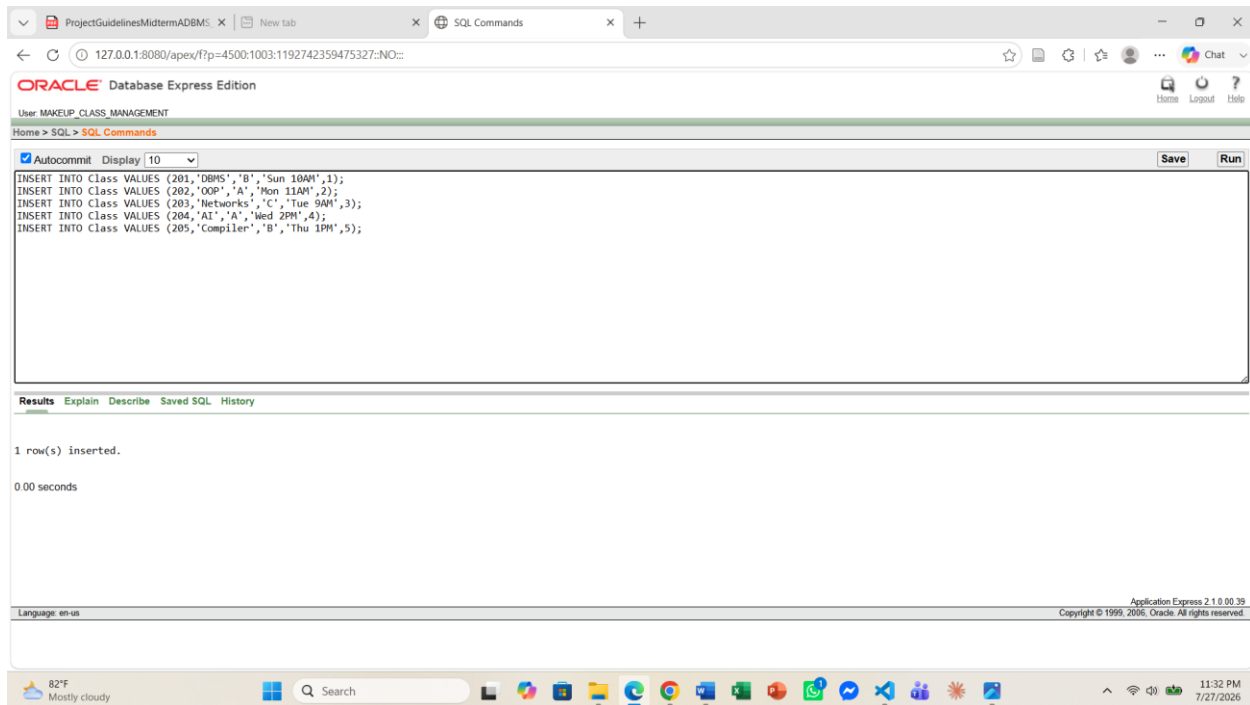
INSERT INTO TeacherPhone VALUES (4,'01744444444');

INSERT INTO TeacherPhone VALUES (5,'01755555555');



5.

```
INSERT INTO Class VALUES (201,'DBMS','B','Sun 10AM',1);  
INSERT INTO Class VALUES (202,'OOP','A','Mon 11AM',2);  
INSERT INTO Class VALUES (203,'Networks','C','Tue 9AM',3);  
INSERT INTO Class VALUES (204,'AI','A','Wed 2PM',4);  
INSERT INTO Class VALUES (205,'Compiler','B','Thu 1PM',5);
```



6.

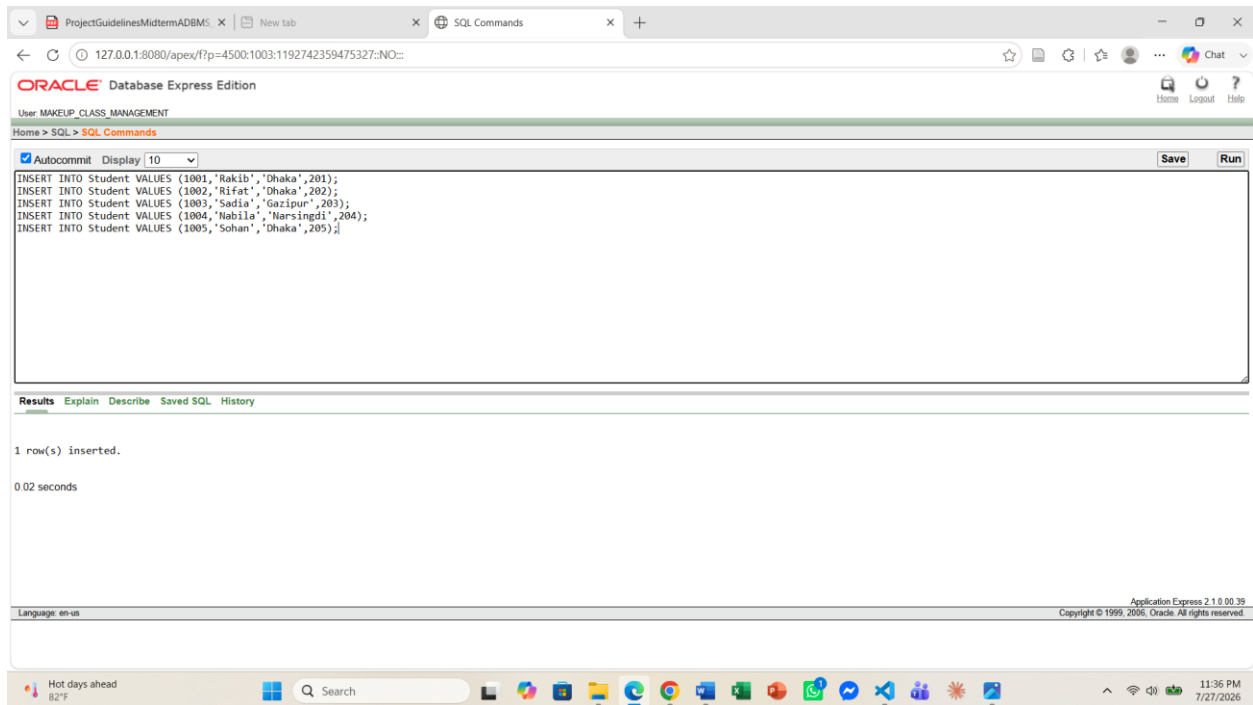
INSERT INTO Student VALUES (1001,'Rakib','Dhaka',201);

INSERT INTO Student VALUES (1002,'Rifat','Dhaka',202);

INSERT INTO Student VALUES (1003,'Sadia','Gazipur',203);

INSERT INTO Student VALUES (1004,'Nabila','Narsingdi',204);

INSERT INTO Student VALUES (1005,'Sohan','Dhaka',205);



7.

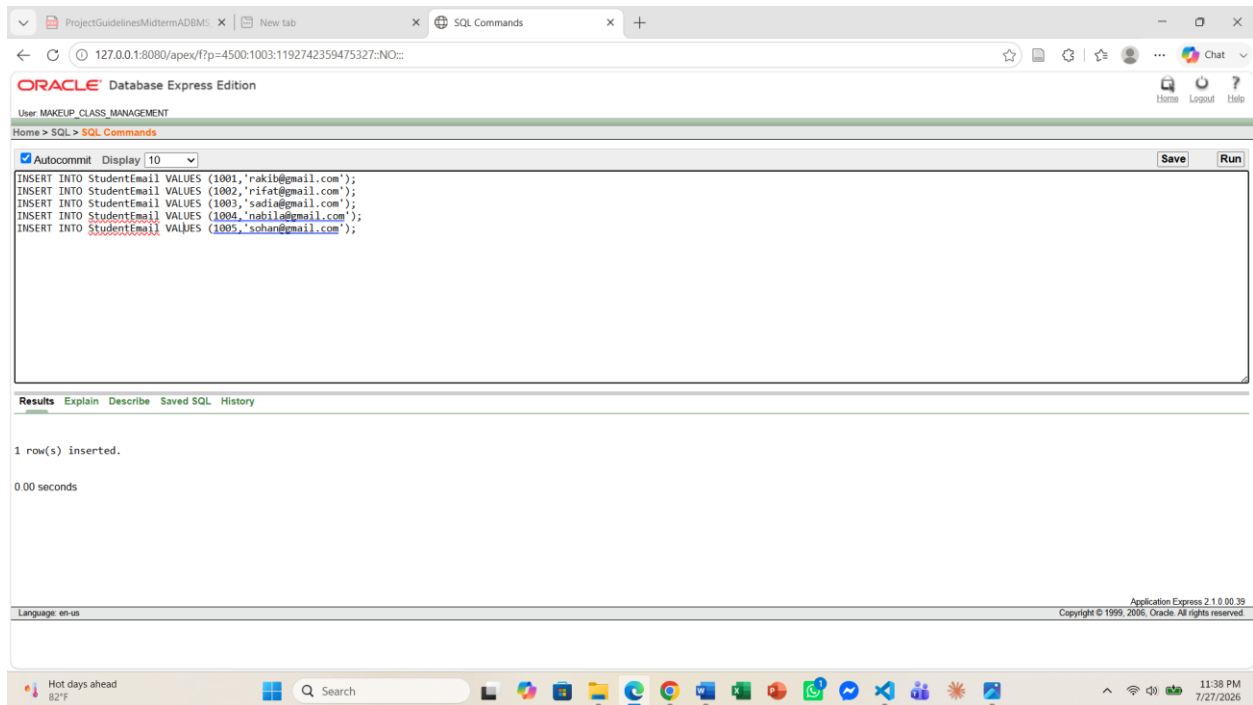
```
INSERT INTO StudentEmail VALUES (1001,'rakib@gmail.com');
```

```
INSERT INTO StudentEmail VALUES (1002,'rifat@gmail.com');
```

```
INSERT INTO StudentEmail VALUES (1003,'sadia@gmail.com');
```

```
INSERT INTO StudentEmail VALUES (1004,'nabila@gmail.com');
```

```
INSERT INTO StudentEmail VALUES (1005,'sohan@gmail.com');
```



8.

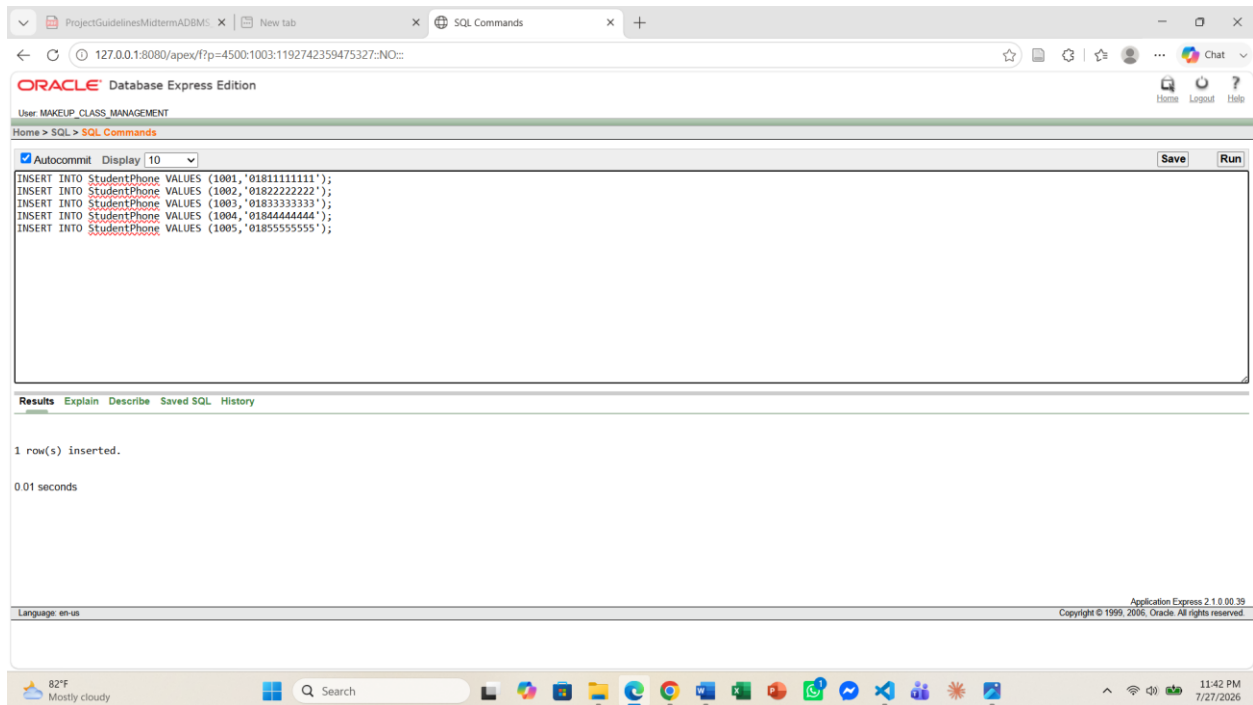
```
INSERT INTO StudentPhone VALUES (1001,'01811111111');
```

```
INSERT INTO StudentPhone VALUES (1002,'01822222222');
```

```
INSERT INTO StudentPhone VALUES (1003,'01833333333');
```

```
INSERT INTO StudentPhone VALUES (1004,'01844444444');
```

```
INSERT INTO StudentPhone VALUES (1005,'01855555555');
```



9.

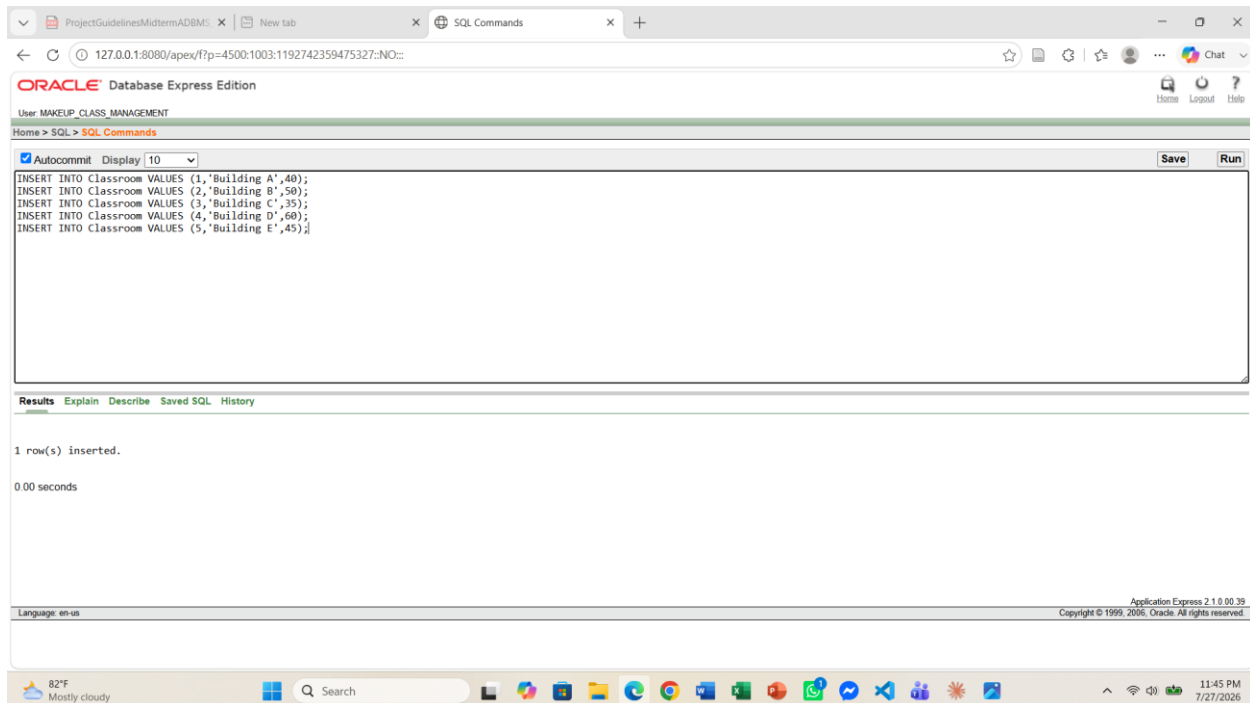
INSERT INTO Classroom VALUES (1,'Building A',40);

INSERT INTO Classroom VALUES (2,'Building B',50);

INSERT INTO Classroom VALUES (3,'Building C',35);

INSERT INTO Classroom VALUES (4,'Building D',60);

INSERT INTO Classroom VALUES (5,'Building E',45);



10.

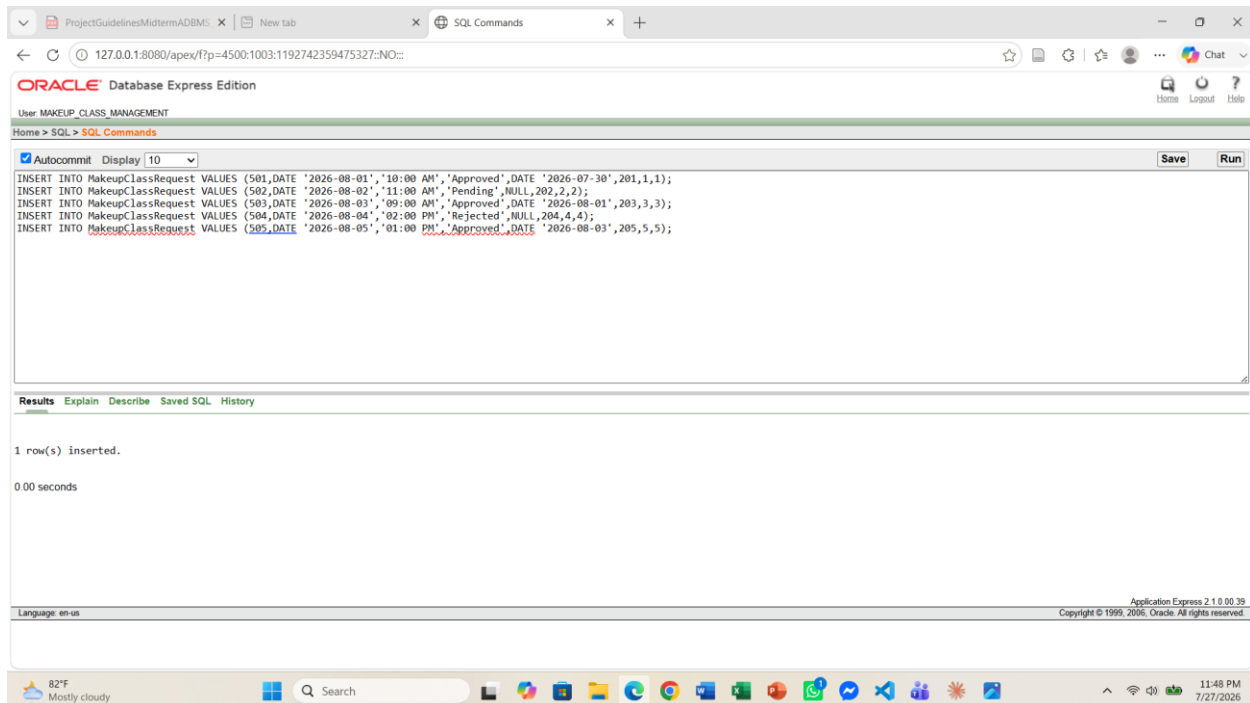
```
INSERT INTO MakeupClassRequest VALUES (501,DATE '2026-08-01','10:00 AM','Approved',DATE '2026-07-30',201,1,1);
```

```
INSERT INTO MakeupClassRequest VALUES (502,DATE '2026-08-02','11:00 AM','Pending',NULL,202,2,2);
```

```
INSERT INTO MakeupClassRequest VALUES (503,DATE '2026-08-03','09:00 AM','Approved',DATE '2026-08-01',203,3,3);
```

```
INSERT INTO MakeupClassRequest VALUES (504,DATE '2026-08-04','02:00 PM','Rejected',NULL,204,4,4);
```

```
INSERT INTO MakeupClassRequest VALUES (505,DATE '2026-08-05','01:00 PM','Approved',DATE '2026-08-03',205,5,5);
```



11.

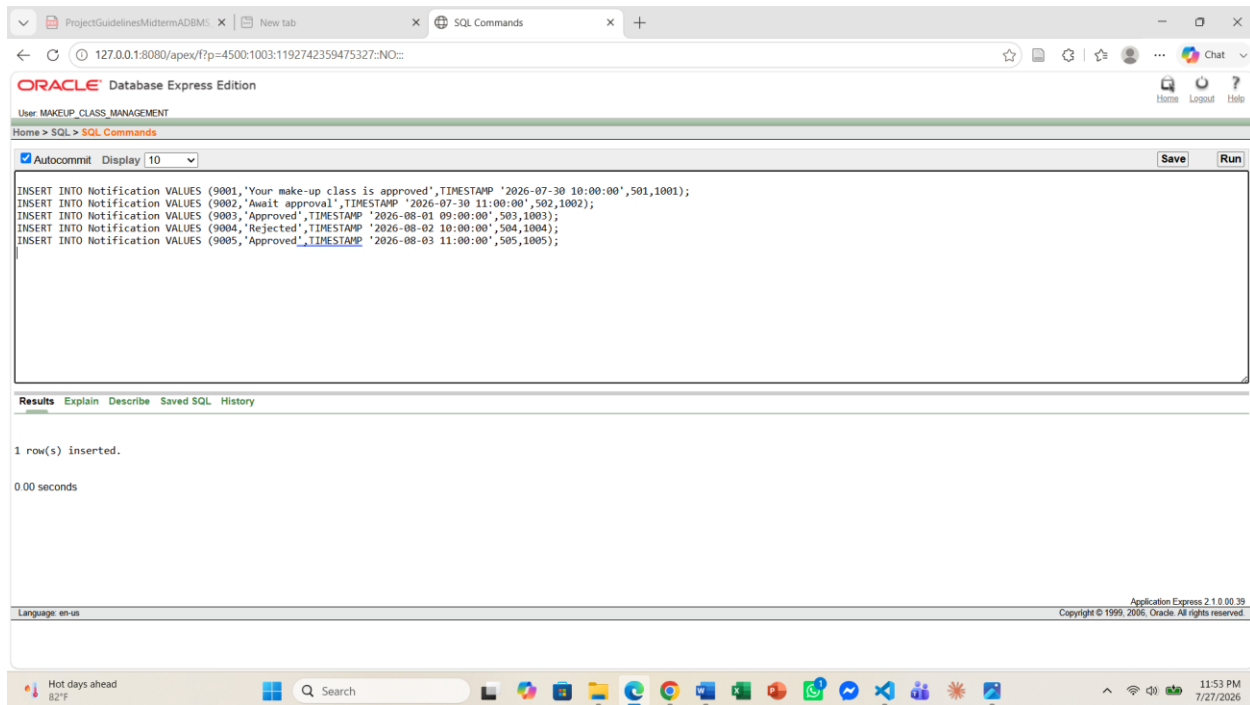
INSERT INTO Notification VALUES (9001,'Your make-up class is approved',TIMESTAMP '2026-07-30 10:00:00',501,1001);

INSERT INTO Notification VALUES (9002,'Await approval',TIMESTAMP '2026-07-30 11:00:00',502,1002);

INSERT INTO Notification VALUES (9003,'Approved',TIMESTAMP '2026-08-01 09:00:00',503,1003);

INSERT INTO Notification VALUES (9004,'Rejected',TIMESTAMP '2026-08-02 10:00:00',504,1004);

INSERT INTO Notification VALUES (9005,'Approved',TIMESTAMP '2026-08-03 11:00:00',505,1005);



Basic PL/SQL:

1. Variables (2)

Query 1: Store Teacher name:

DECLARE

 v_teacher_name VARCHAR2(50) := 'Anis';

BEGIN

 DBMS_OUTPUT.PUT_LINE('Teacher Name: ' || v_teacher_name);

END;

/

The screenshot shows the Oracle Database Express Edition interface. The browser tabs include 'New tab', 'SQL Commands', and 'ProjectGuidelinesMidtermADBM:'. The address bar shows the URL '127.0.0.1:8080/apex/f?p=4500:1003:3133697148232467::NO::'. The page title is 'ORACLE Database Express Edition'. The user is 'MAKEUP_CLASS_MANAGEMENT'. The breadcrumb navigation is 'Home > SQL > SQL Commands'. The SQL editor has 'Autocommit' checked and 'Display' set to '10'. The SQL code is as follows:

```
/*Project name: Make-up class management system
course name: Advance Database Management System
Semester: Summer
Section: B
*/
DECLARE
  v_teacher_name VARCHAR2(50) := 'Anis';
BEGIN
  DBMS_OUTPUT.PUT_LINE('Teacher Name: ' || v_teacher_name);
END;
/
```

The results section shows the output: 'Teacher Name: Anis', 'Statement processed.', and '0.02 seconds'. The footer indicates 'Application Express 2.1.0.00.19' and 'Copyright © 1999, 2006, Oracle. All rights reserved.'.

Query 2: Store Classroom Capacity

DECLARE

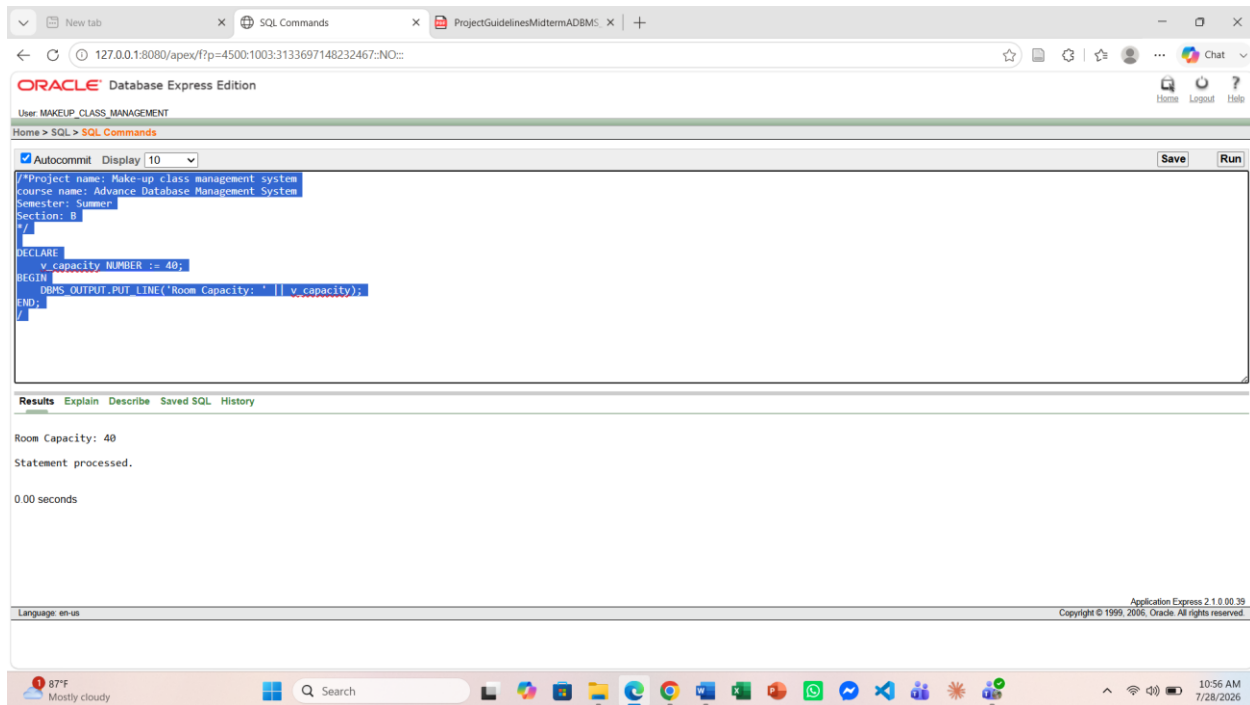
 v_capacity NUMBER := 40;

BEGIN

 DBMS_OUTPUT.PUT_LINE('Room Capacity: ' || v_capacity);

END;

/



2. Operators (2)

Query 1: Arithmetic Operator

DECLARE

room1 NUMBER := 40;

room2 NUMBER := 20;

total NUMBER;

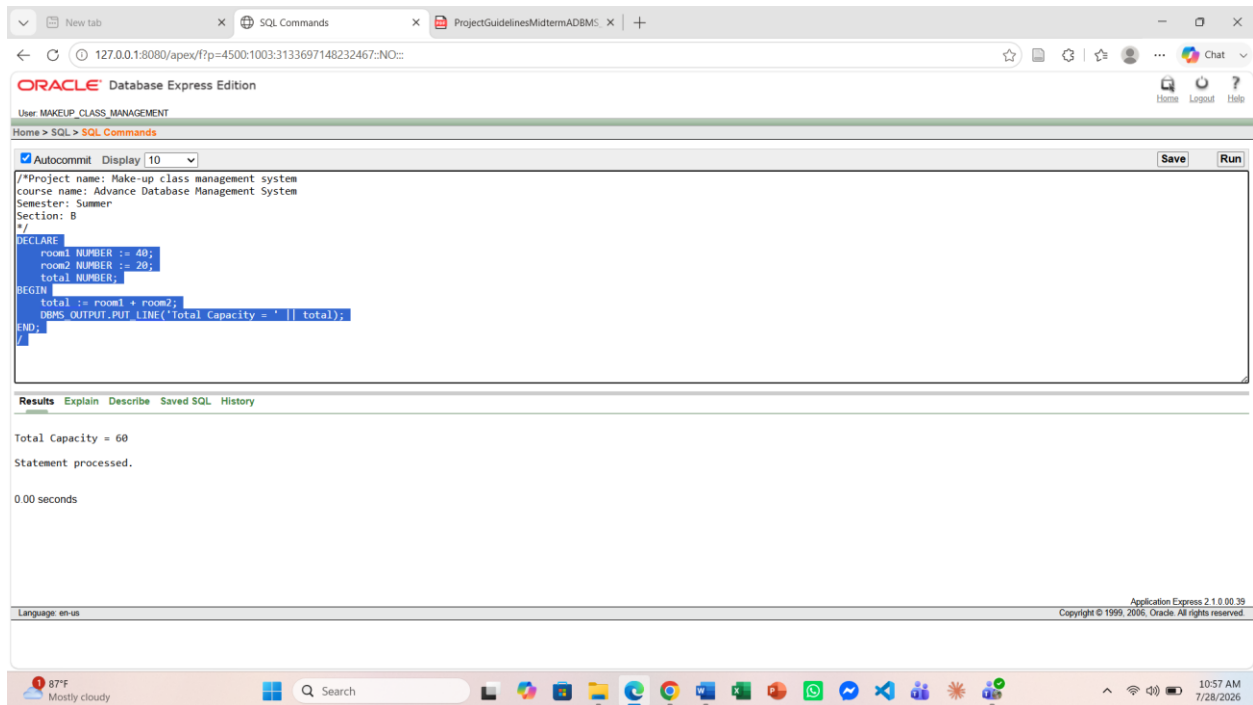
BEGIN

total := room1 + room2;

DBMS_OUTPUT.PUT_LINE('Total Capacity = ' || total);

END;

/



Query 2: Comparison Operator

DECLARE

total_students NUMBER := 35;

BEGIN

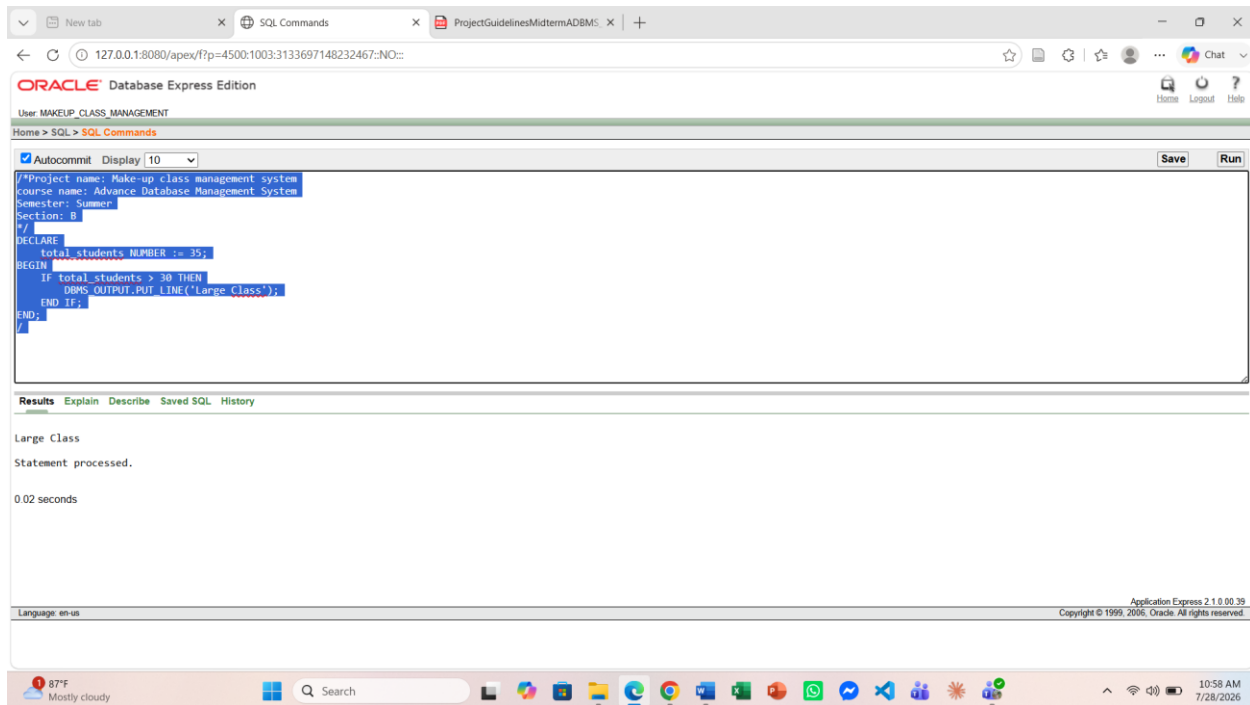
IF total_students > 30 THEN

DBMS_OUTPUT.PUT_LINE('Large Class');

END IF;

END;

/



3. Single-Row Functions (2)

Query 1: Display Teacher Name in Uppercase

DECLARE

 v_name VARCHAR2(50);

BEGIN

 SELECT UPPER(t_name)

 INTO v_name

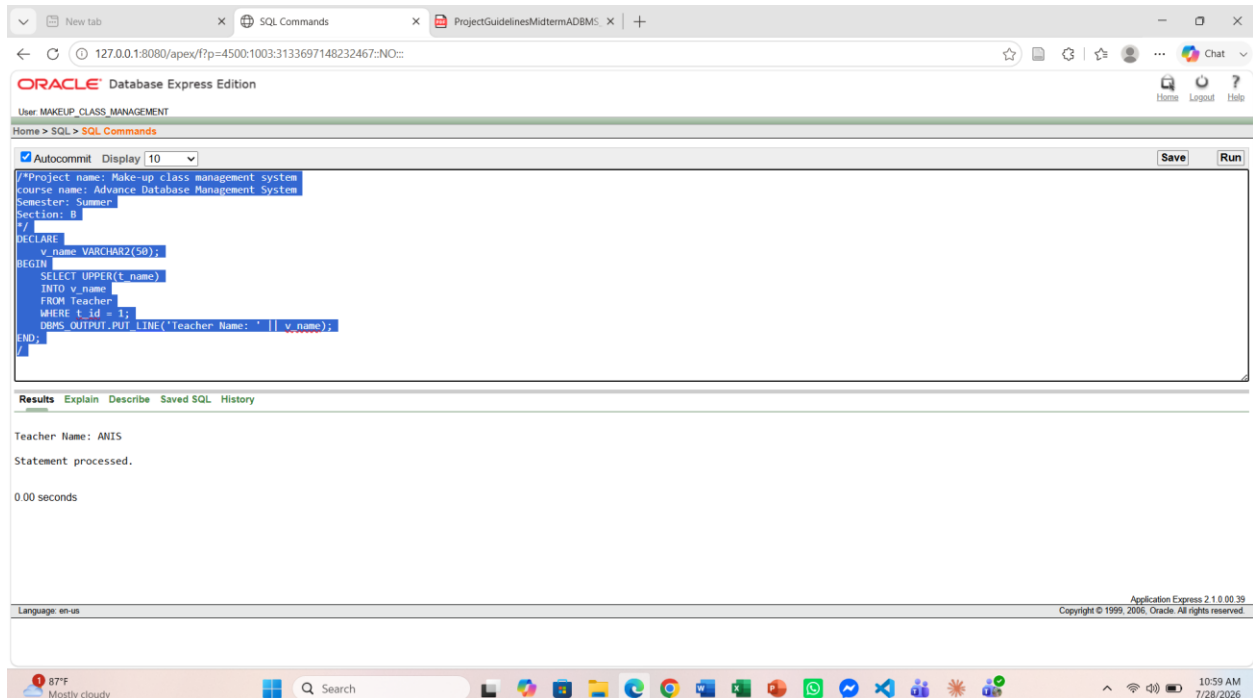
 FROM Teacher

 WHERE t_id = 1;

 DBMS_OUTPUT.PUT_LINE('Teacher Name: ' || v_name);

END;

/



Query 2: Show Length of Student Name

DECLARE

 v_length NUMBER;

BEGIN

 SELECT LENGTH(s_name)

 INTO v_length

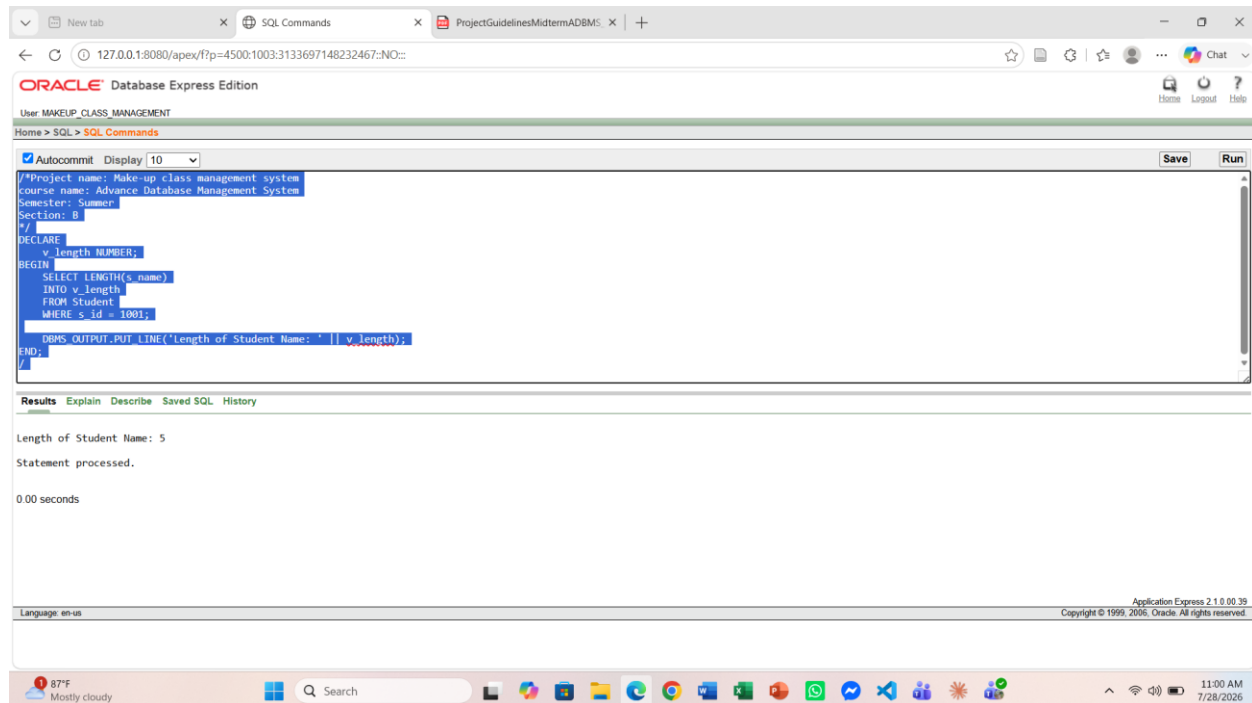
 FROM Student

 WHERE s_id = 1001;

 DBMS_OUTPUT.PUT_LINE('Length of Student Name: ' || v_length);

END;

/



4. Group Function

Query 1: COUNT()

Question: Write a PL/SQL program to count the total number of students.

DECLARE

 v_total NUMBER;

BEGIN

 SELECT COUNT(*)

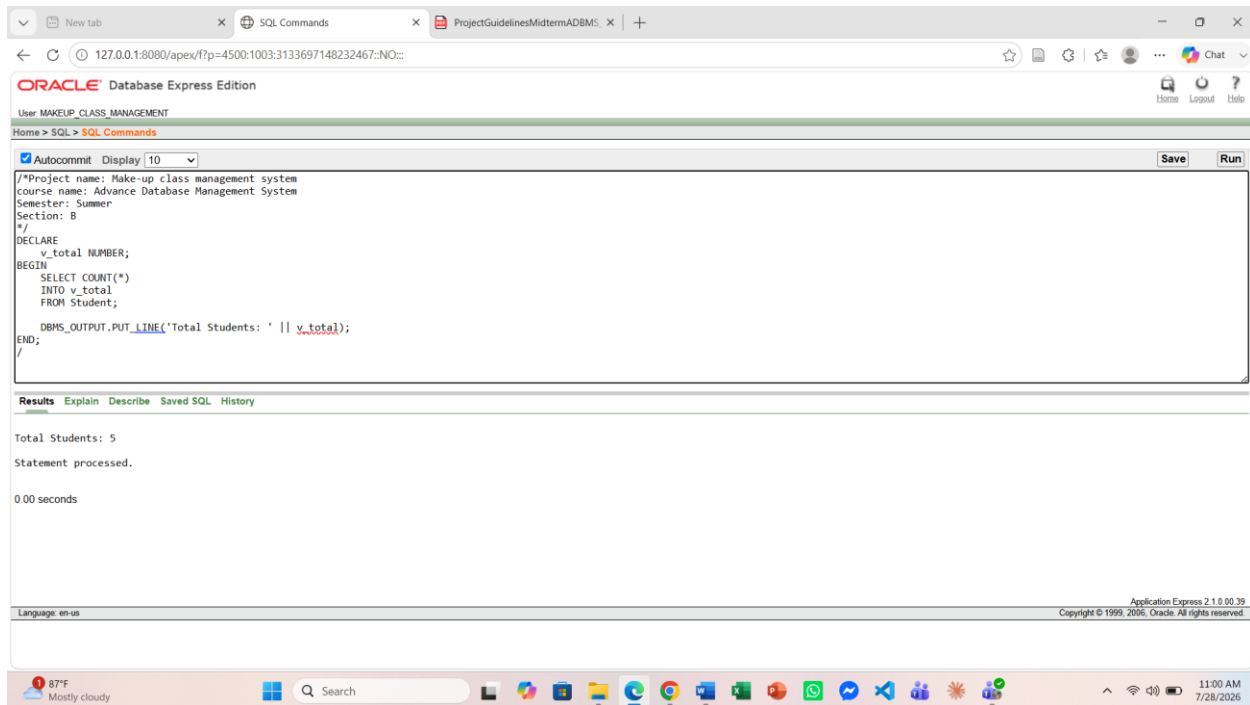
 INTO v_total

 FROM Student;

 DBMS_OUTPUT.PUT_LINE('Total Students: ' || v_total);

END;

/



Query 2: AVG()

Question: Write a PL/SQL program to calculate the average classroom capacity.

DECLARE

 v_avg NUMBER;

BEGIN

 SELECT AVG(capacity)

 INTO v_avg

 FROM Classroom;

 DBMS_OUTPUT.PUT_LINE('Average Classroom Capacity: ' || v_avg);

END;

/

The screenshot shows the Oracle Database Express Edition interface. The browser address bar indicates the URL: 127.0.0.1:8080/apex/f?p=4500:1003:3133697148232467::NO::. The page title is "ORACLE Database Express Edition". The user is logged in as "User MAKEUP_CLASS_MANAGEMENT". The "SQL Commands" window is active, showing a SQL script with a comment block and a query to calculate the average classroom capacity. The script is as follows:

```
/*Project name: Make-up class management system
course name: Advance Database Management System
Semester: Summer
Section: B
*/
DECLARE
  v_avg NUMBER;
BEGIN
  SELECT AVG(capacity)
  INTO v_avg
  FROM Classroom;
  DBMS_OUTPUT.PUT_LINE('Average Classroom Capacity: ' || v_avg);
END;
```

The "Results" tab is selected, showing the output of the query: "Average Classroom Capacity: 46". Below this, it states "Statement processed." and "0.00 seconds". The bottom status bar shows "Language: en-us" and "Application Express 2.1.0.00.19 Copyright © 1999, 2006, Oracle. All rights reserved."

5. Loop (2)

Query 1: FOR Loop

BEGIN

FOR i IN 1..5 LOOP

DBMS_OUTPUT.PUT_LINE('Make-up Class ' || i);

END LOOP;

END;

/

The screenshot shows the Oracle Database Express Edition interface. The browser address bar indicates the URL: 127.0.0.1:8080/apex/f?p=4500:1003:3133697148232467::NO::. The page title is "ORACLE Database Express Edition". The user is logged in as "User MAKEUP_CLASS_MANAGEMENT". The breadcrumb navigation shows "Home > SQL > SQL Commands". The SQL editor has "Autocommit" checked and "Display" set to 10. The SQL code entered is:

```
/*Project name: Make-up class management system
course name: Advance Database Management System
Semester: Summer
Section: 8
*/
BEGIN
  FOR i IN 1..5 LOOP
    DBMS_OUTPUT.PUT_LINE('Make-up Class ' || i);
  END LOOP;
END;
```

The "Results" tab is selected, showing the output of the query:

```
Make-up Class 1
Make-up Class 2
Make-up Class 3
Make-up Class 4
Make-up Class 5

Statement processed.

0.00 seconds
```

The footer of the application shows "Application Express 2.1.0.00.39" and "Copyright © 1999, 2006, Oracle. All rights reserved." The Windows taskbar at the bottom shows the date and time as 11:02 AM 7/28/2026.

Query 2: WHILE Loop

DECLARE

 i NUMBER := 1;

BEGIN

 WHILE i <= 5 LOOP

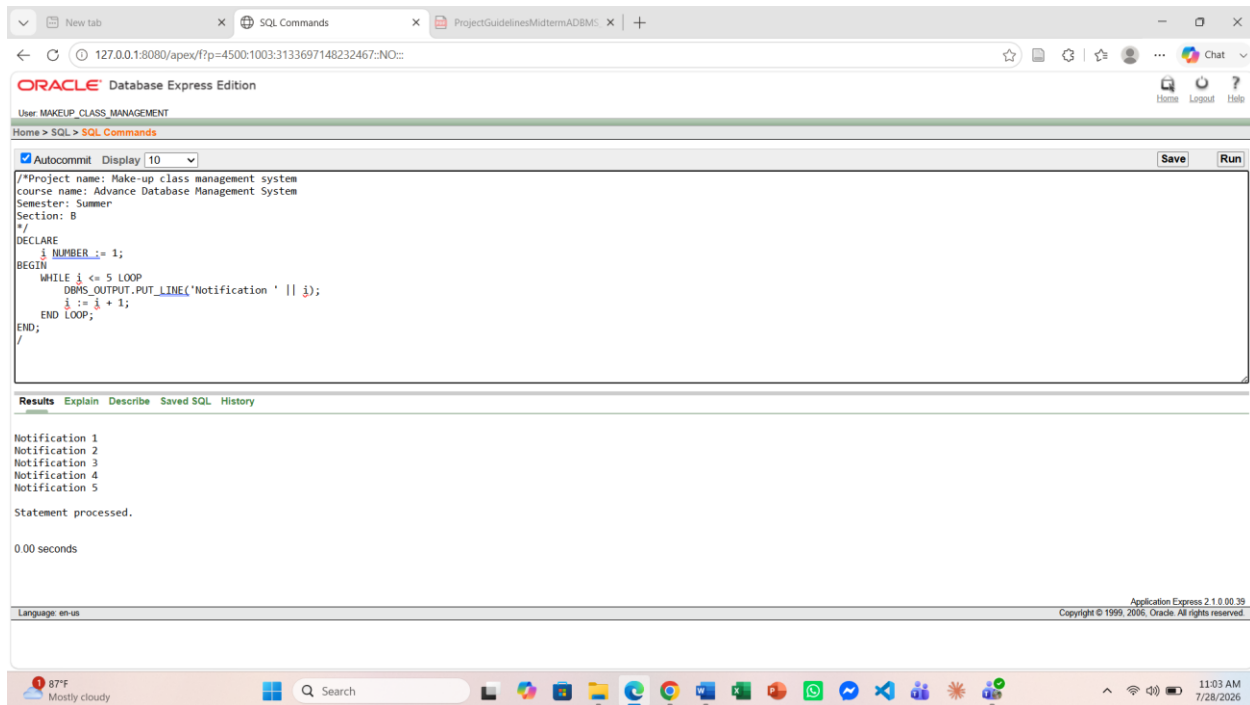
 DBMS_OUTPUT.PUT_LINE('Notification ' || i);

 i := i + 1;

 END LOOP;

END;

/



6. Conditional Statements (2)

Query 1: IF Statement

DECLARE

 v_status VARCHAR2(20) := 'Approved';

BEGIN

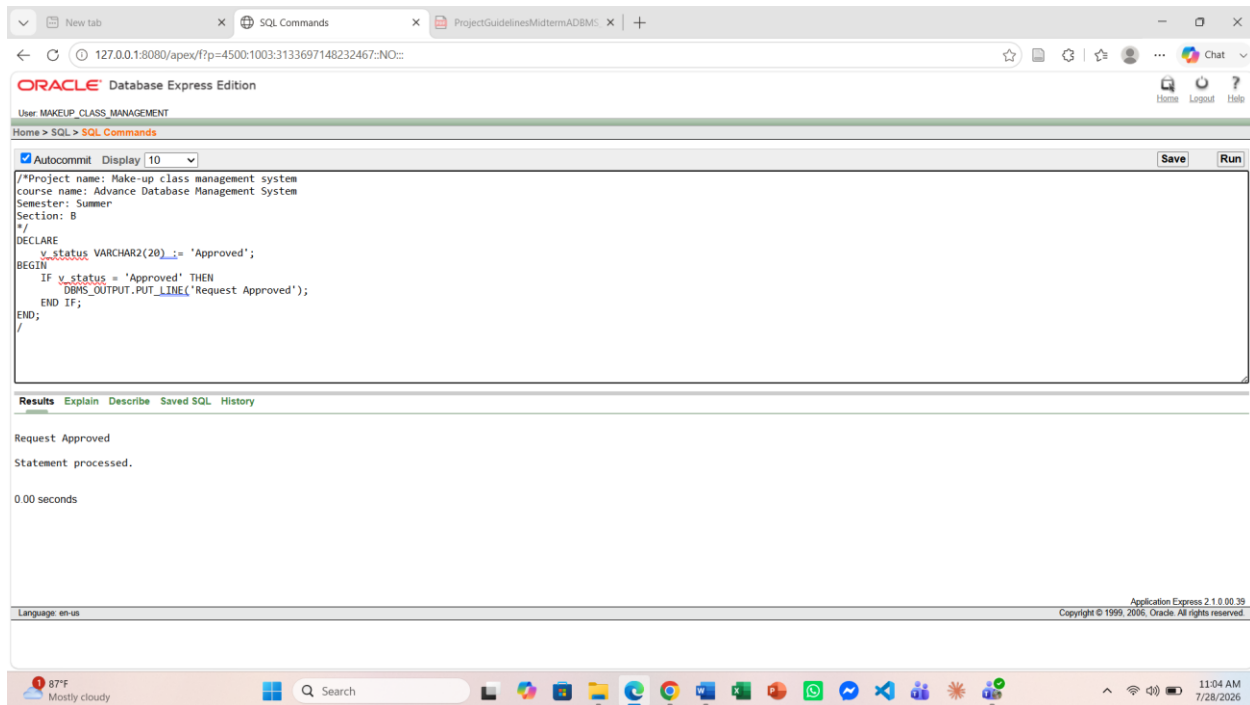
 IF v_status = 'Approved' THEN

 DBMS_OUTPUT.PUT_LINE('Request Approved');

 END IF;

END;

/



Query 2: IF...ELSIF...ELSE

DECLARE

 v_capacity NUMBER := 45;

BEGIN

 IF v_capacity >= 60 THEN

 DBMS_OUTPUT.PUT_LINE('Large Room');

 ELSIF v_capacity >= 40 THEN

 DBMS_OUTPUT.PUT_LINE('Medium Room');

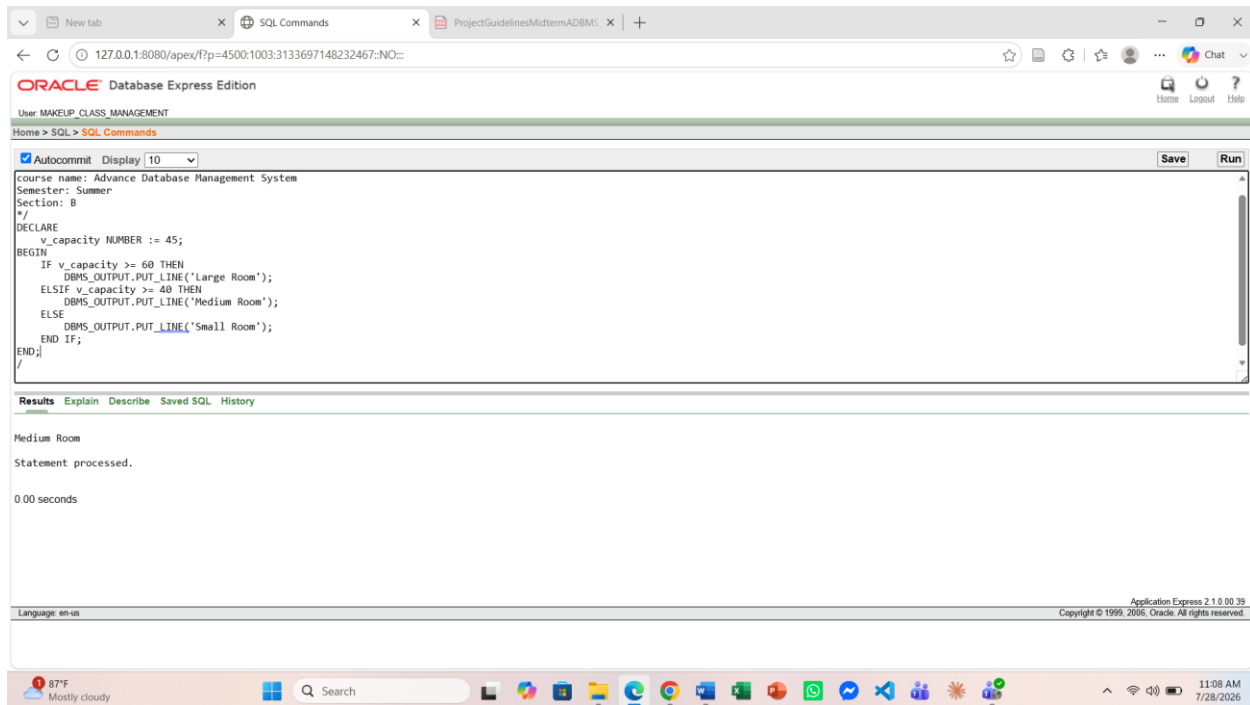
 ELSE

 DBMS_OUTPUT.PUT_LINE('Small Room');

 END IF;

END;

/



7. Subquery (2)

Query 1: Teacher of the DBMS Class

Question: Write a PL/SQL program using a subquery to display the teacher name of the DBMS class.

DECLARE

 v_teacher Teacher.t_name%TYPE;

BEGIN

 SELECT t_name

 INTO v_teacher

 FROM Teacher

 WHERE t_id = (

 SELECT t_id

 FROM Class

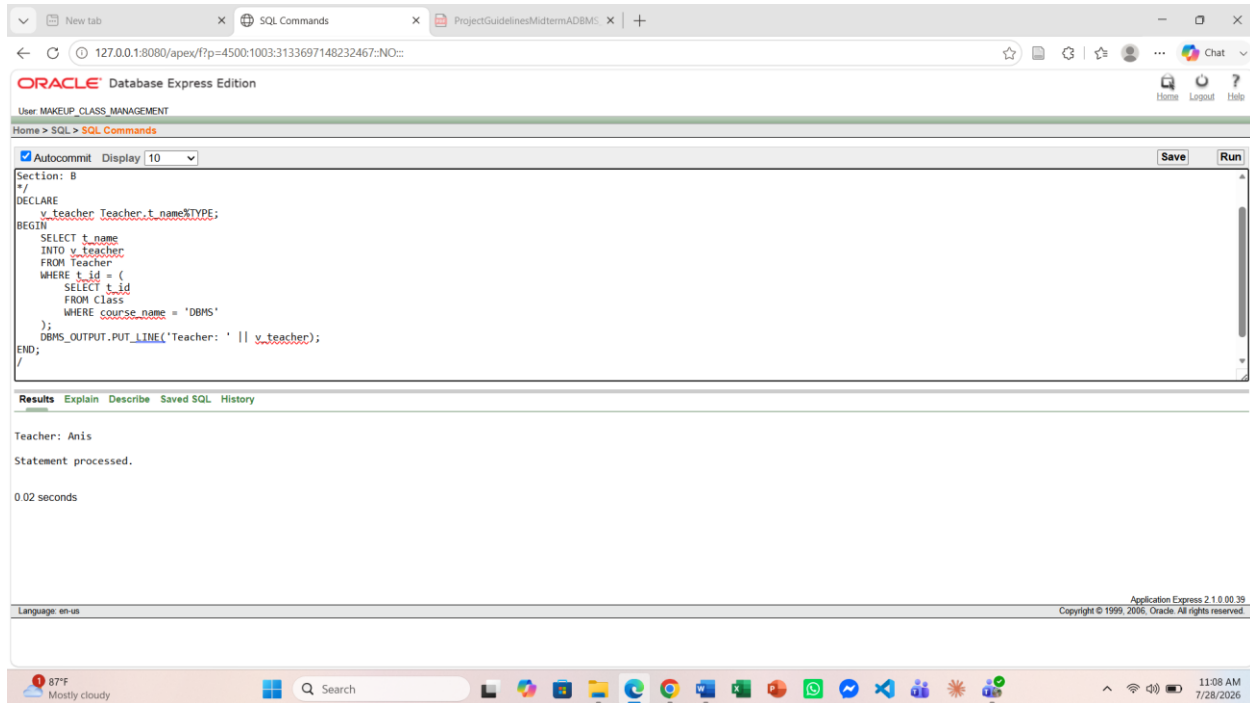
 WHERE course_name = 'DBMS'

);

 DBMS_OUTPUT.PUT_LINE('Teacher: ' || v_teacher);

END;

/



Query 2: Students of the DBMS Class

Question: Write a PL/SQL program using a subquery to display the names of students enrolled in the DBMS class.

DECLARE

CURSOR c_student IS

SELECT s_name

FROM Student

WHERE class_id = (

SELECT class_id FROM Class

WHERE course_name = 'DBMS'

);

v_name Student.s_name%TYPE;

BEGIN

OPEN c_student;

LOOP

FETCH c_student INTO v_name;

EXIT WHEN c_student%NOTFOUND;

```

        DBMS_OUTPUT.PUT_LINE(v_name);

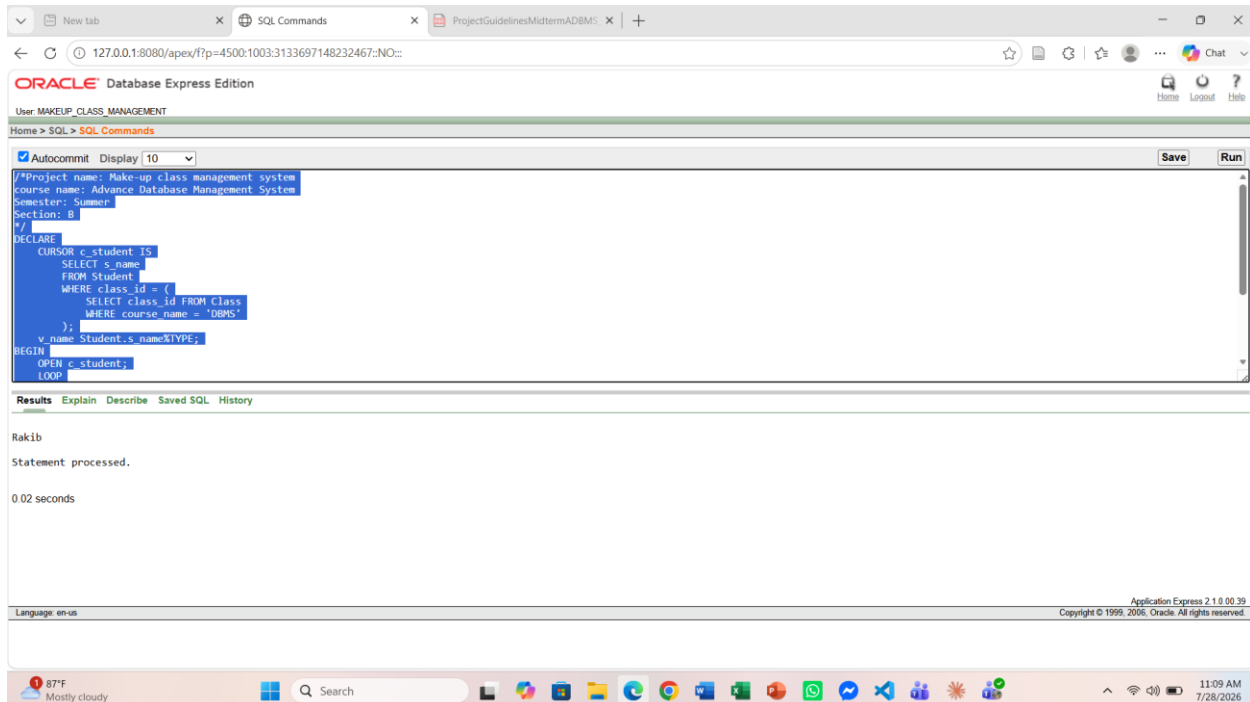
    END LOOP;

    CLOSE c_student;

END;

/

```



8. Joining (2)

Query 1: Teacher with Department

Question: Write a PL/SQL program using JOIN to display each teacher's name with the department name.

DECLARE

CURSOR c_teacher IS

SELECT t.t_name, d.dept_name

FROM Teacher t

JOIN Department d

ON t.dept_id = d.dept_id;

v_tname Teacher.t_name%TYPE;

v_dept Department.dept_name%TYPE;

BEGIN

OPEN c_teacher;

LOOP

FETCH c_teacher INTO v_tname, v_dept;

EXIT WHEN c_teacher%NOTFOUND;

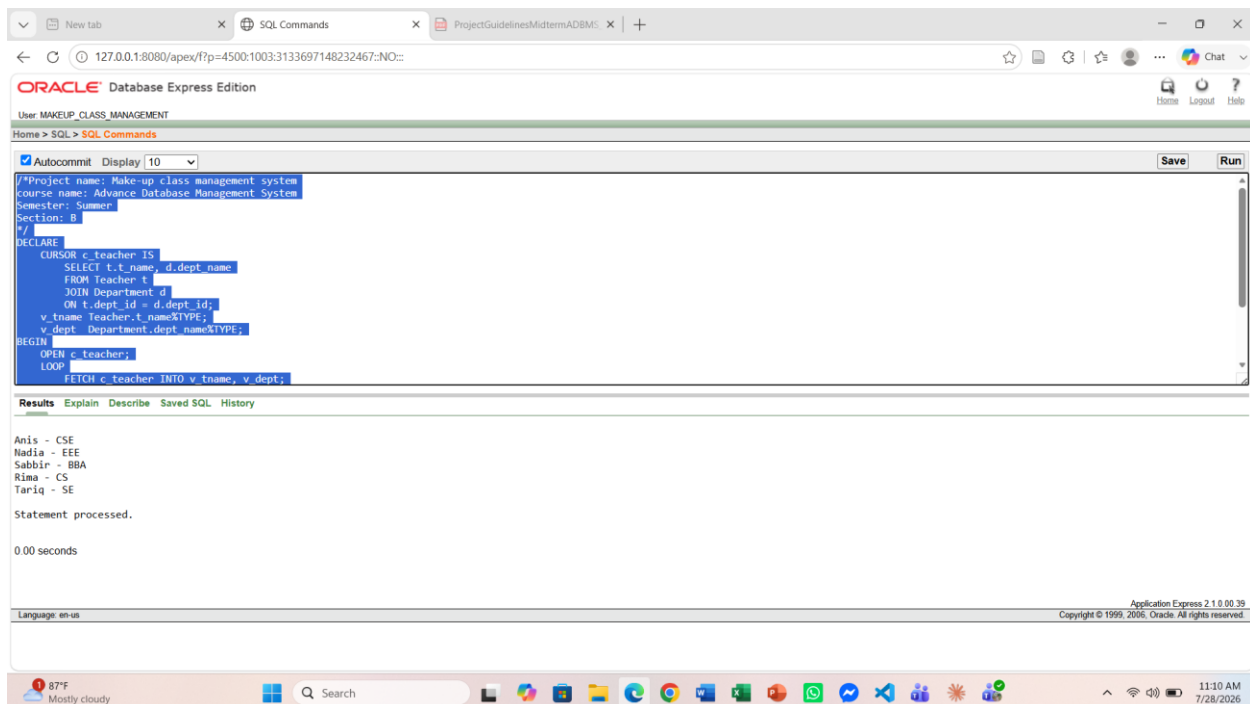
DBMS_OUTPUT.PUT_LINE(v_tname || ' - ' || v_dept);

END LOOP;

CLOSE c_teacher;

END;

/



Query 2: Student with Class

Question: Write a PL/SQL program using JOIN to display the student name, course name, and section.

DECLARE

CURSOR c_student IS

SELECT s.s_name, c.course_name, c.section

FROM Student s

JOIN Class c

```

        ON s.class_id = c.class_id;

v_name  Student.s_name%TYPE;

v_course Class.course_name%TYPE;

v_section Class.section%TYPE;

BEGIN

    OPEN c_student;

    LOOP

        FETCH c_student INTO v_name, v_course, v_section;

        EXIT WHEN c_student%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE(v_name || ' - ' ||      v_course || ' - ' ||      v_section);

    END LOOP;

    CLOSE c_student;

END;

/

```

The screenshot shows the Oracle Database Express Edition web interface. The browser address bar displays the URL: 127.0.0.1:8080/apex/f?p=4500:1003:3133697148232467::NO::. The page title is "ORACLE Database Express Edition". The user is logged in as "User MAKEUP_CLASS_MANAGEMENT". The "SQL Commands" tab is active, showing the following SQL script:

```

/*Project name: Make-up class management system
course name: Advance Database Management System
Semester: Summer
Section: B
*/
DECLARE
CURSOR c_student IS
    SELECT s.s_name, c.course_name, c.section
    FROM Student s
    JOIN Class c
    ON s.class_id = c.class_id;
v_name  Student.s_name%TYPE;
v_course Class.course_name%TYPE;
v_section Class.section%TYPE;
BEGIN
    OPEN c_student;
    LOOP

```

The "Results" tab is active, displaying the output of the script:

```

Rakib - DBMS - B
Rifat - OOP - A
Sadla - Networks - C
Nabila - AI - A
Sohan - Compiler - B

```

The status "Statement processed." and "0.01 seconds" are shown. The footer indicates "Application Express 2.1.0.00.39 Copyright © 1999, 2006, Oracle. All rights reserved." and "Language: en-us". The Windows taskbar at the bottom shows the system clock as 11:11 AM on 7/28/2026.

Advance PL/SQL:

1. Stored Function

Question 1

Write a PL/SQL stored function to return the name of a student using the Student ID.

```
CREATE OR REPLACE FUNCTION get_student_name(p_sid NUMBER)
```

```
RETURN VARCHAR2
```

```
IS
```

```
    v_name Student.s_name%TYPE;
```

```
BEGIN
```

```
    SELECT s_name
```

```
    INTO v_name
```

```
    FROM Student
```

```
    WHERE s_id = p_sid;
```

```
    RETURN v_name;
```

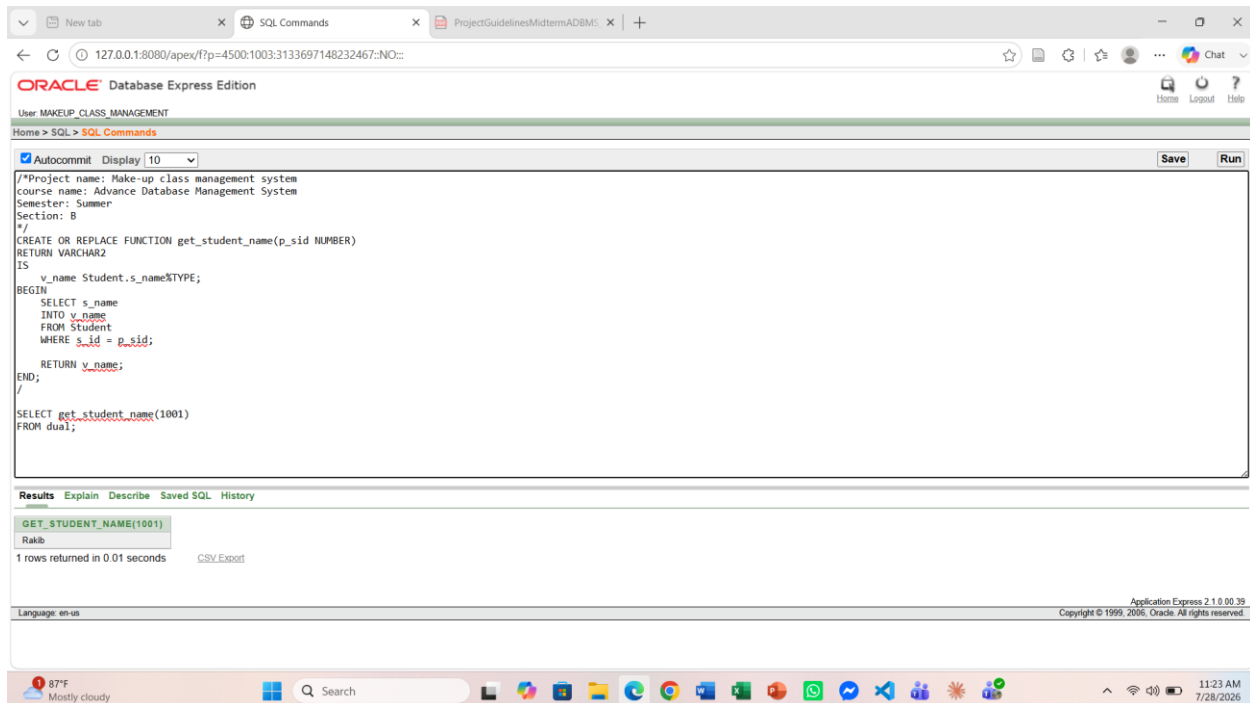
```
END;
```

```
/
```

Call the function

```
SELECT get_student_name(1001)
```

```
FROM dual;
```



Question 2

Write a PL/SQL stored function to return the name of a teacher using the Teacher ID.

```
CREATE OR REPLACE FUNCTION get_teacher_name(p_tid NUMBER)
```

```
RETURN VARCHAR2
```

```
IS
```

```
    v_name Teacher.t_name%TYPE;
```

```
BEGIN
```

```
    SELECT t_name
```

```
    INTO v_name
```

```
    FROM Teacher
```

```
    WHERE t_id = p_tid;
```

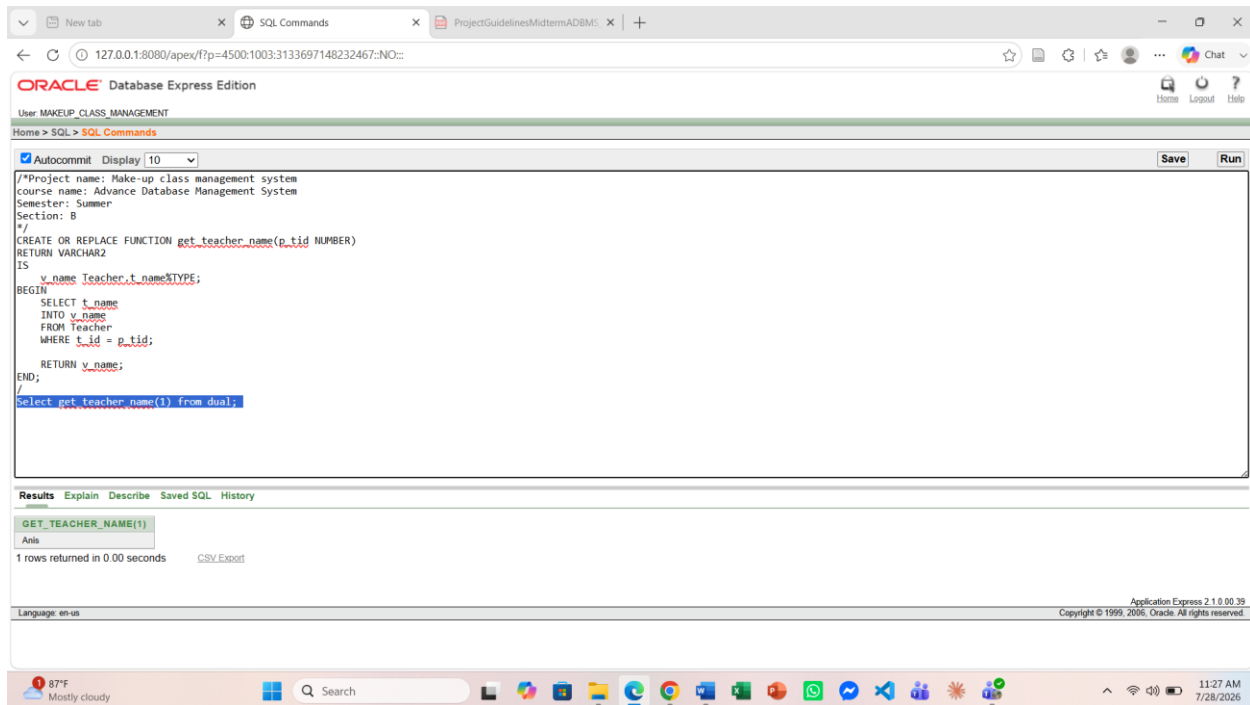
```
    RETURN v_name;
```

```
END;
```

```
/
```

Call the function

```
Select get_teacher_name(1) from dual;
```



2. Stored Procedure

Question 1

Write a PL/SQL stored procedure to display the name of a student using Student ID.

CREATE OR REPLACE PROCEDURE show_student(p_sid NUMBER)

IS

 v_name Student.s_name%TYPE;

BEGIN

 SELECT s_name

 INTO v_name

 FROM Student

 WHERE s_id = p_sid;

 DBMS_OUTPUT.PUT_LINE(v_name);

END;

/

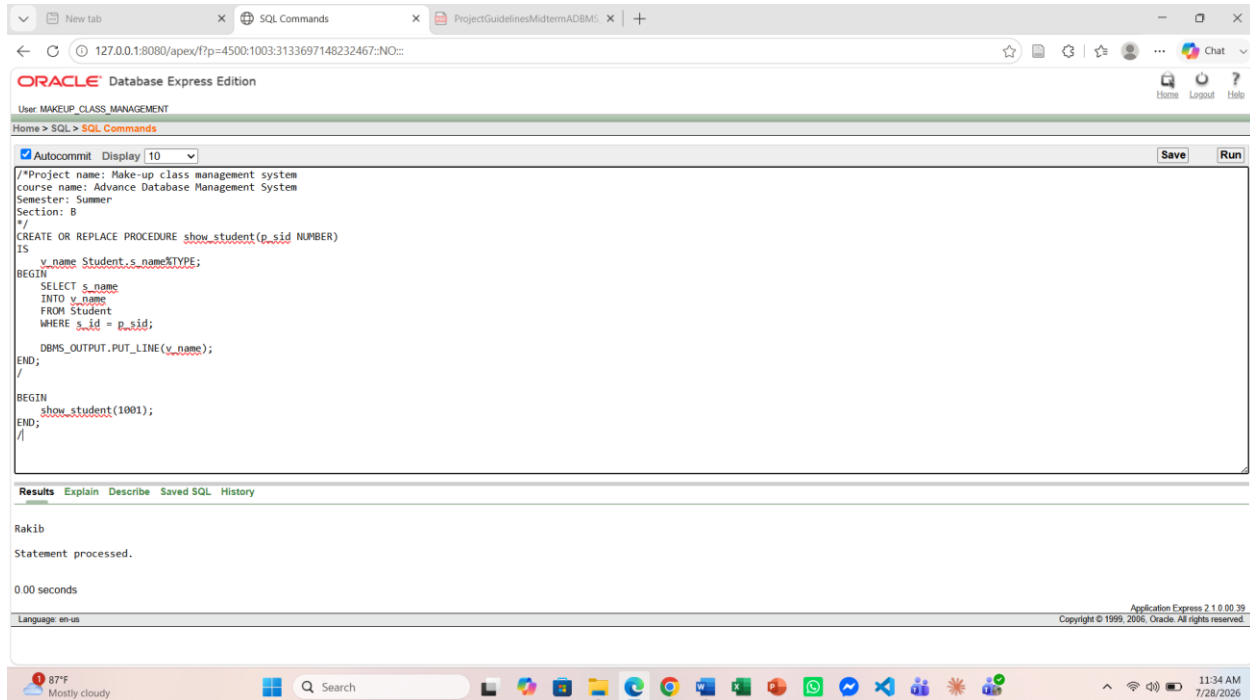
Execute:

BEGIN

```
show_student(1001);
```

```
END;
```

```
/
```



Question 2

Write a PL/SQL stored procedure to update the status of a Make-up Class Request to 'Approved' using Request ID.

```
CREATE OR REPLACE PROCEDURE update_status(p_req NUMBER)
```

```
IS
```

```
BEGIN
```

```
    UPDATE MakeupClassRequest
```

```
    SET status='Approved'
```

```
    WHERE req_id=p_req;
```

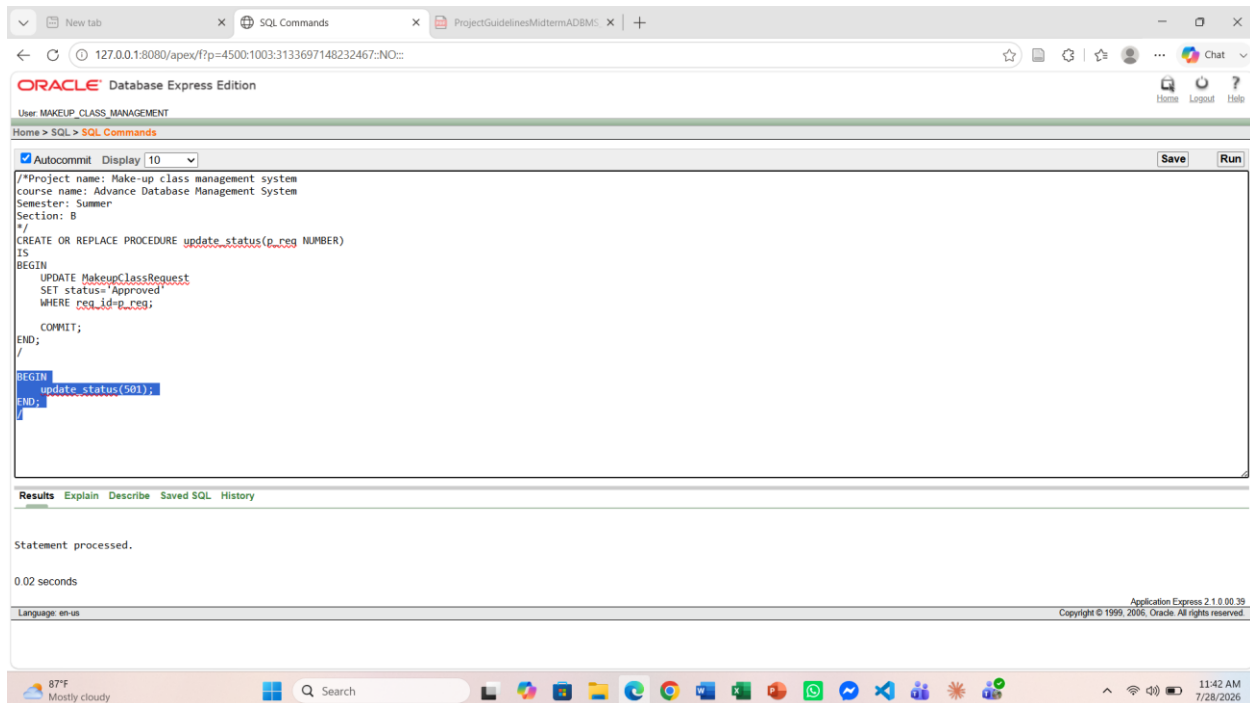
```
    COMMIT;
```

```
END;
```

```
/
```

```
BEGIN
```

```
    update_status(501); END; /
```



3. Table-Based Record

Question 1

Write a PL/SQL program using a table-based record (%ROWTYPE) to retrieve and display a teacher's information.

DECLARE

 v_teacher Teacher%ROWTYPE;

BEGIN

 SELECT *

 INTO v_teacher

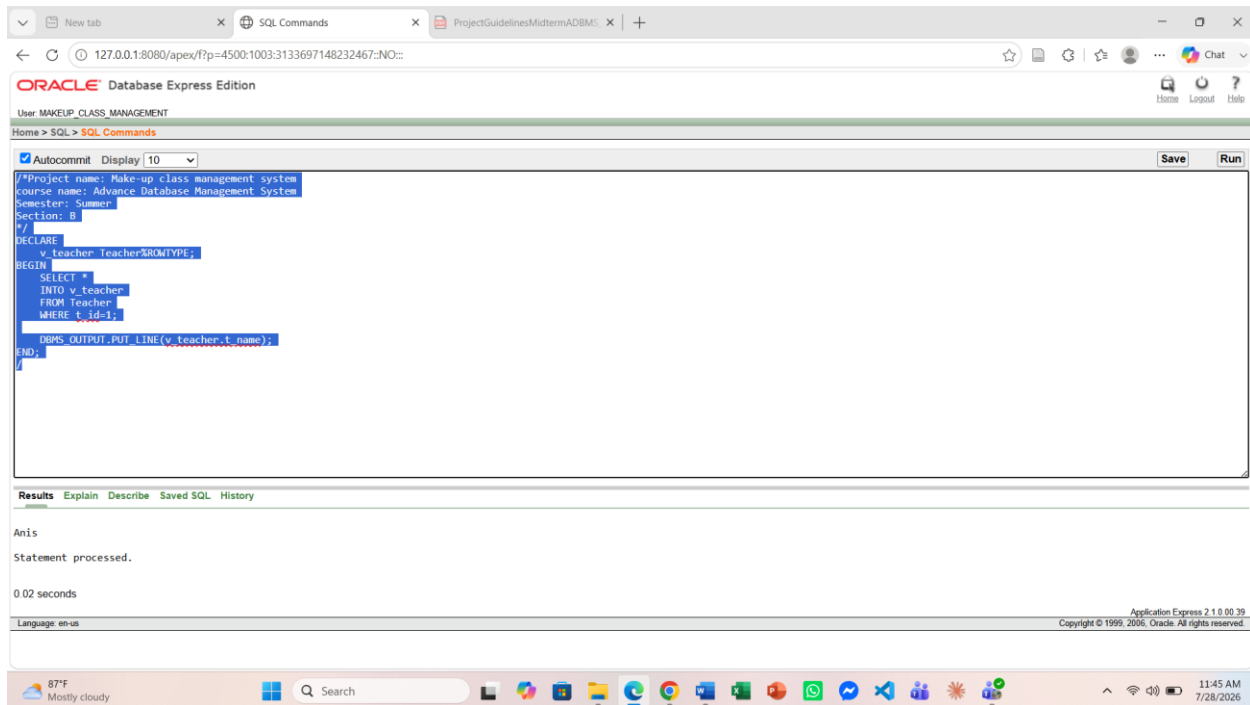
 FROM Teacher

 WHERE t_id=1;

 DBMS_OUTPUT.PUT_LINE(v_teacher.t_name);

END;

/



Question 2

Write a PL/SQL program using a table-based record (%ROWTYPE) to retrieve and display a student's information.

DECLARE

 v_student Student%ROWTYPE;

BEGIN

 SELECT *

 INTO v_student

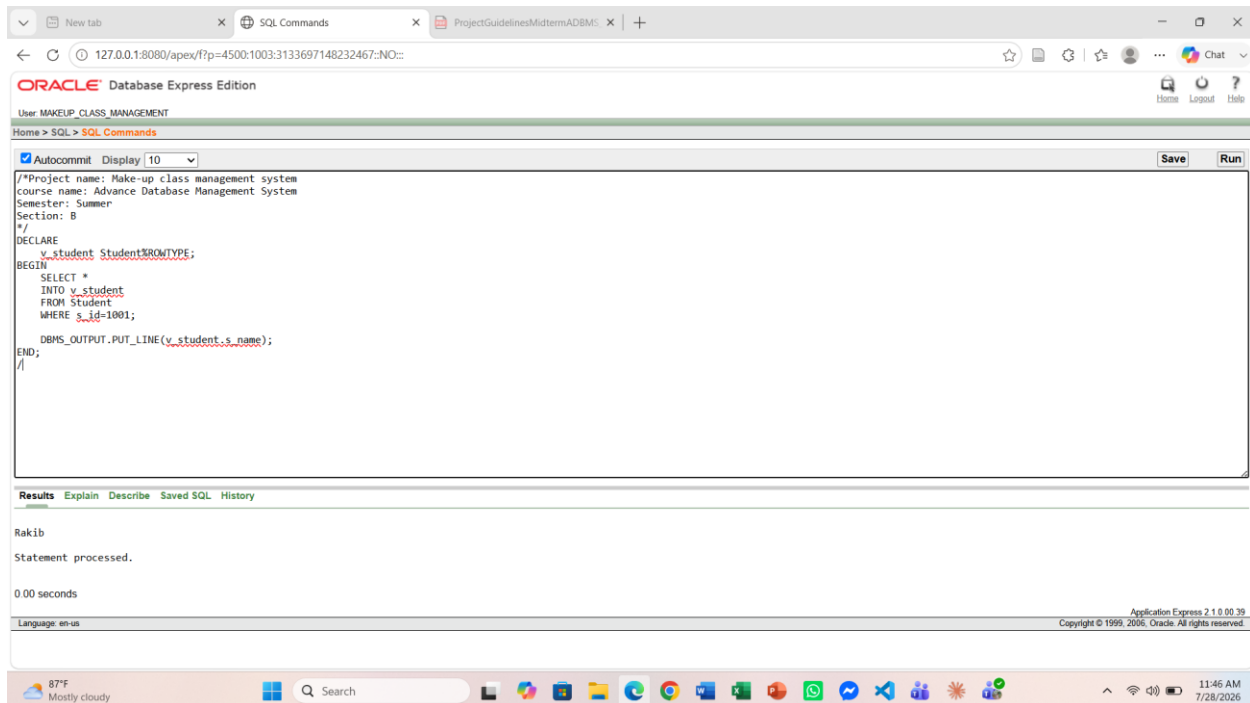
 FROM Student

 WHERE s_id=1001;

 DBMS_OUTPUT.PUT_LINE(v_student.s_name);

END;

/



4. Explicit Cursor

Question 1

Write a PL/SQL program using an explicit cursor to display the names of all students.

DECLARE

CURSOR c IS SELECT s_name FROM Student;

v_name Student.s_name%TYPE;

BEGIN

OPEN c;

LOOP

FETCH c INTO v_name;

EXIT WHEN c%NOTFOUND;

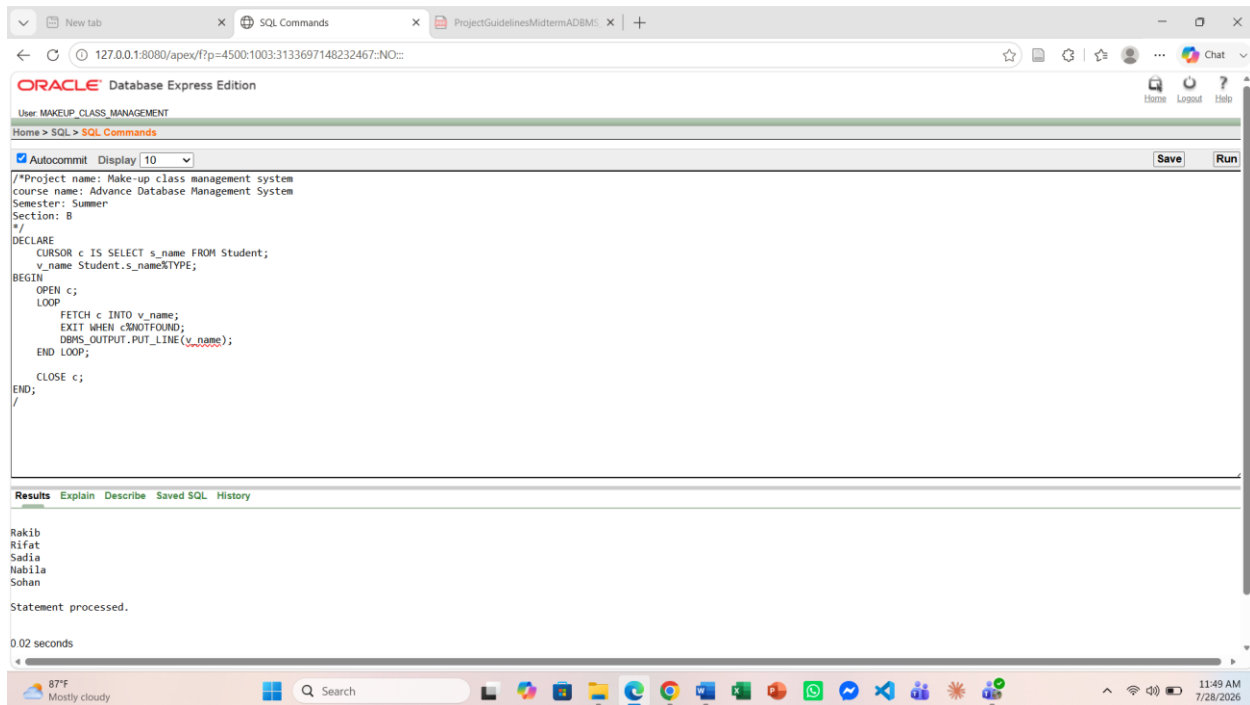
DBMS_OUTPUT.PUT_LINE(v_name);

END LOOP;

CLOSE c;

END;

/



Question 2

Write a PL/SQL program using an explicit cursor to display the names of all teachers.

DECLARE

CURSOR c IS

SELECT t_name FROM Teacher;

v_name Teacher.t_name%TYPE;

BEGIN

OPEN c;

LOOP

FETCH c INTO v_name;

EXIT WHEN c%NOTFOUND;

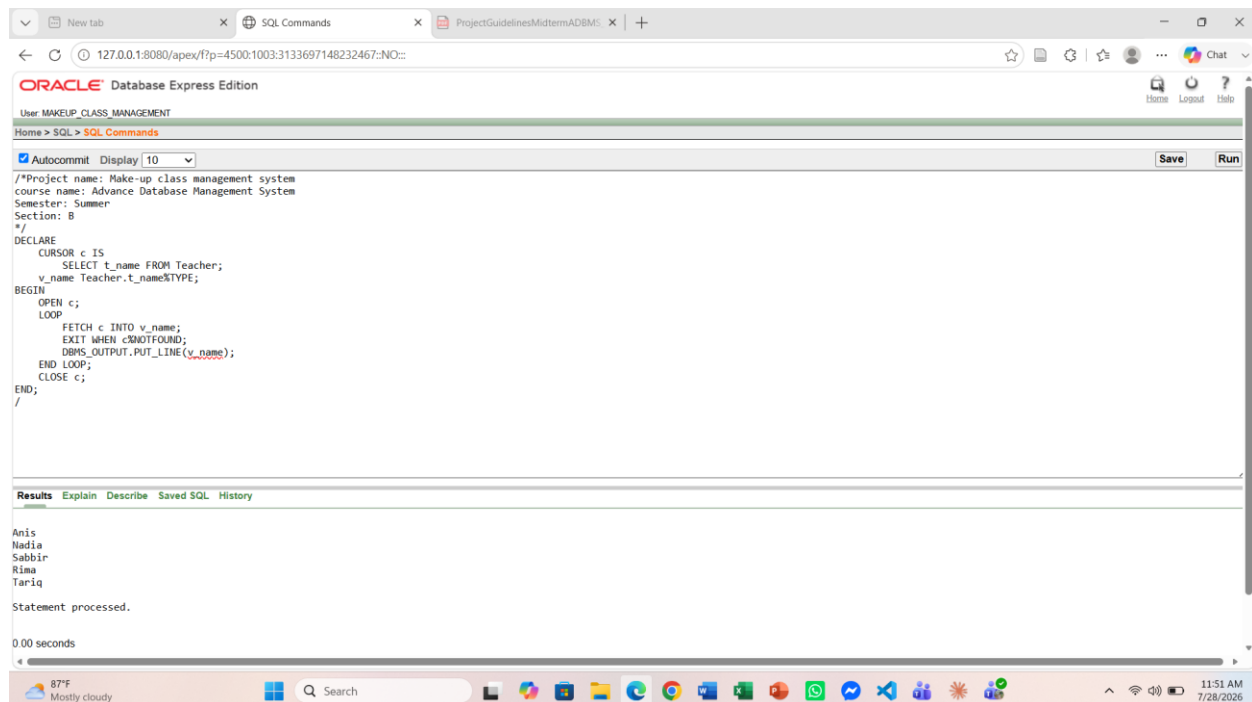
DBMS_OUTPUT.PUT_LINE(v_name);

END LOOP;

CLOSE c;

END;

/



5. Cursor-Based Record

Question 1

Write a PL/SQL program using a cursor-based record to display all student names.

DECLARE

CURSOR c IS

SELECT * FROM Student;

rec c%ROWTYPE;

BEGIN

OPEN c;

LOOP

FETCH c INTO rec;

EXIT WHEN c%NOTFOUND;

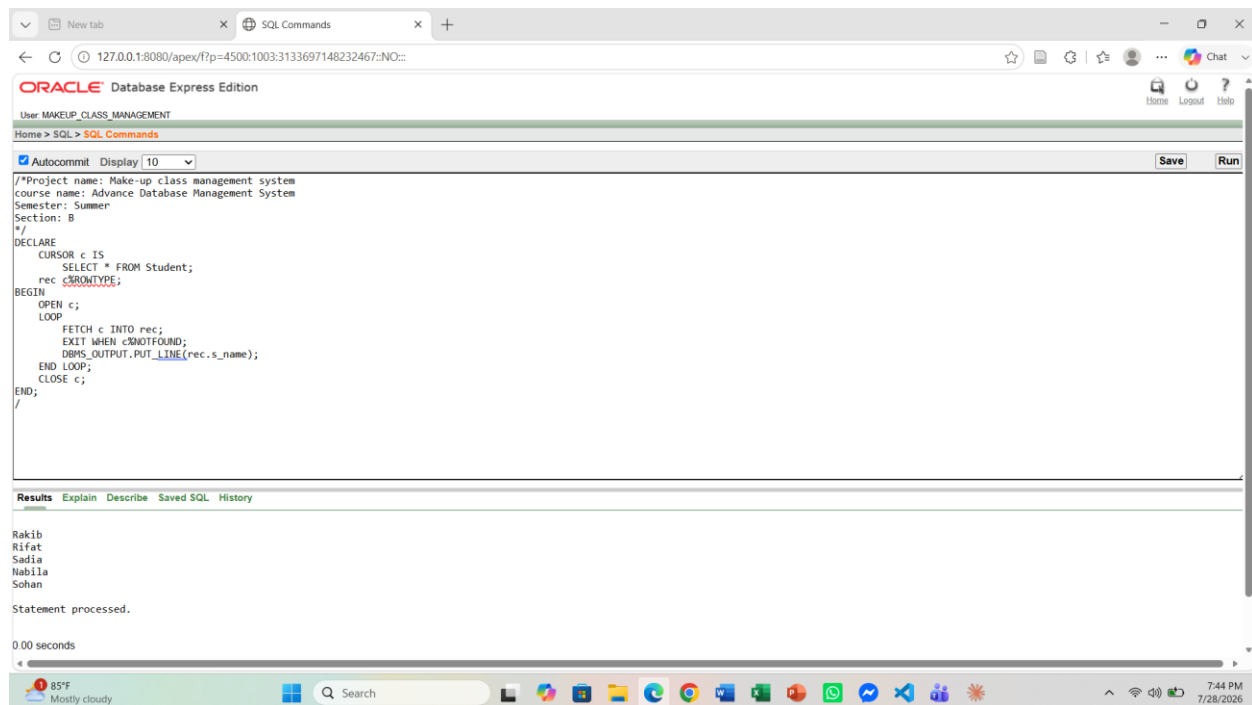
DBMS_OUTPUT.PUT_LINE(rec.s_name);

END LOOP;

CLOSE c;

END;

/



Question 2

Write a PL/SQL program using a cursor-based record to display all teacher names.

DECLARE

CURSOR c IS

SELECT * FROM Teacher;

rec c%ROWTYPE;

BEGIN

OPEN c;

LOOP

FETCH c INTO rec;

EXIT WHEN c%NOTFOUND;

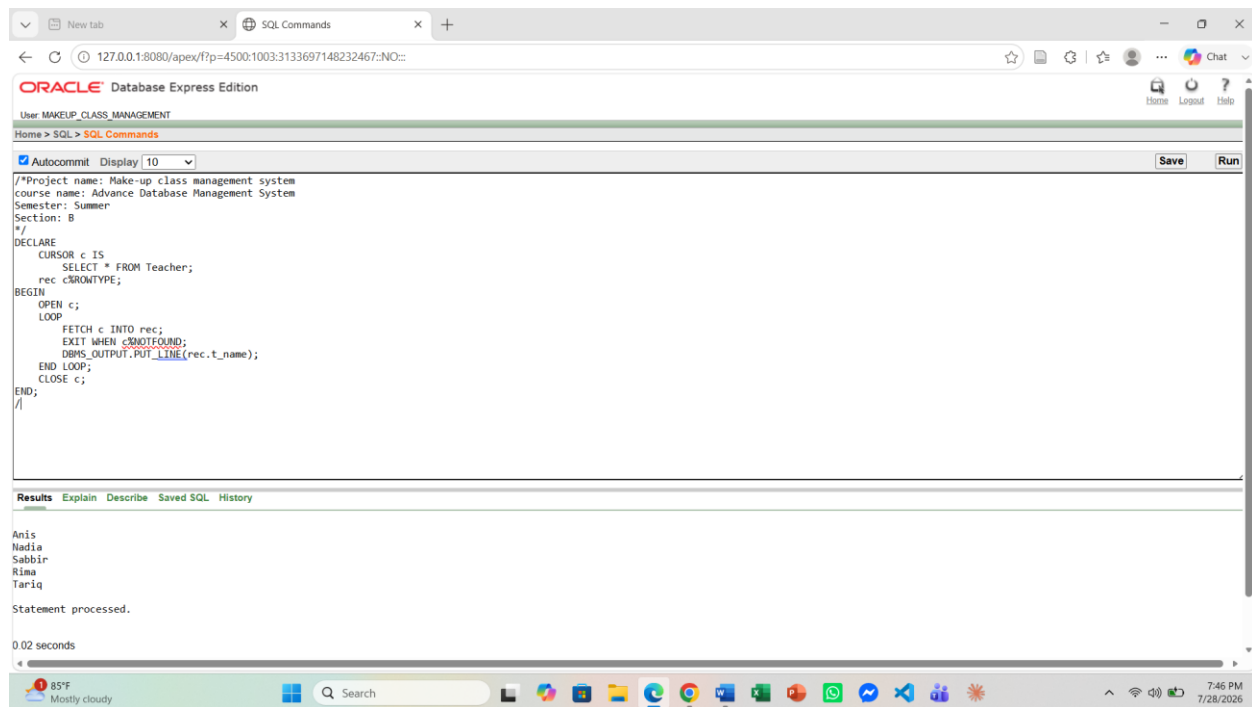
DBMS_OUTPUT.PUT_LINE(rec.t_name);

END LOOP;

CLOSE c;

END;

/



6. Row-Level Trigger

Question 1

Write a row-level trigger that displays a message whenever a new student is inserted into the Student table.

CREATE OR REPLACE TRIGGER trg_student_insert

AFTER INSERT ON Student

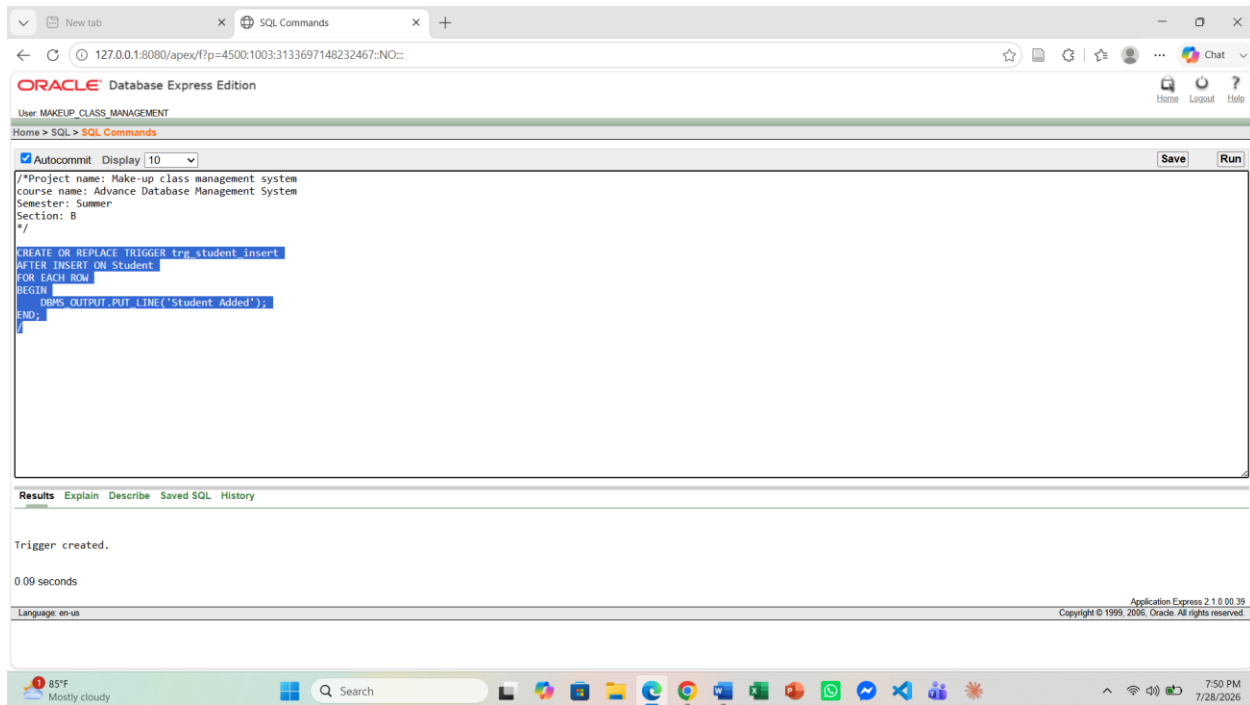
FOR EACH ROW

BEGIN

 DBMS_OUTPUT.PUT_LINE('Student Added');

END;

/



Question 2

Write a row-level trigger that displays a message whenever a teacher record is updated.

CREATE OR REPLACE TRIGGER trg_teacher_update

AFTER UPDATE ON Teacher

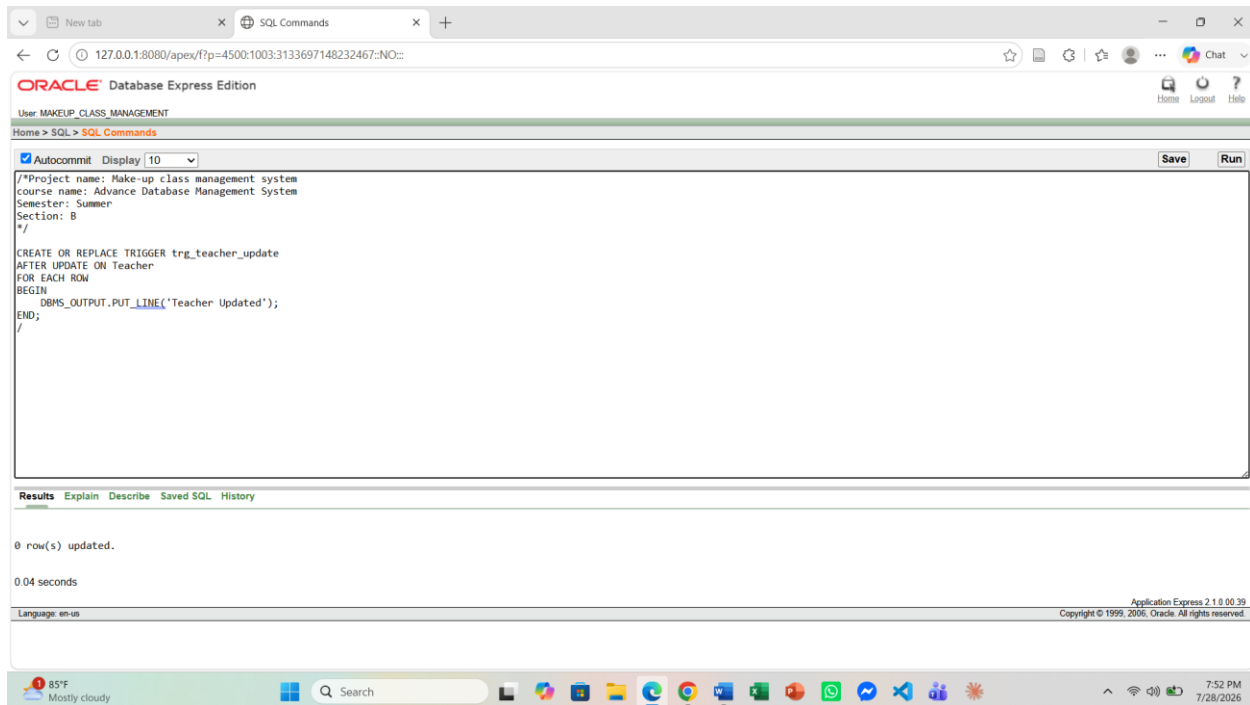
FOR EACH ROW

BEGIN

DBMS_OUTPUT.PUT_LINE('Teacher Updated');

END;

/

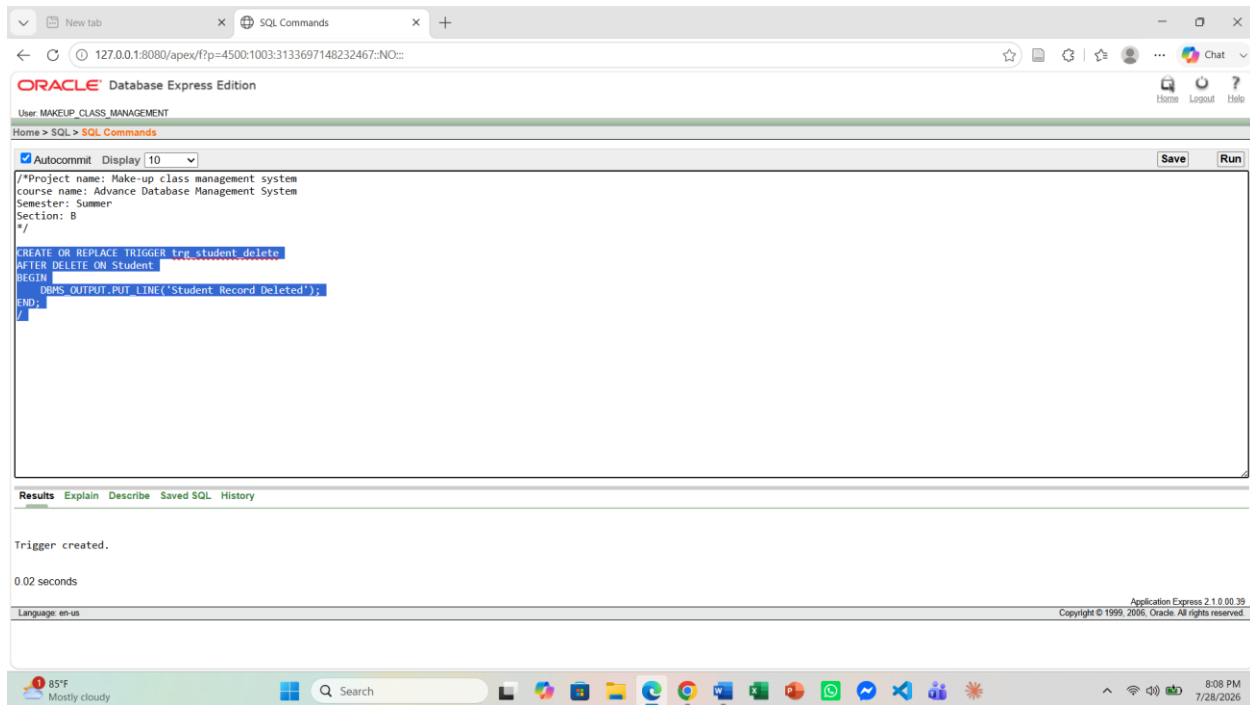


7. Statement-Level Trigger

Question 1

Write a statement-level trigger that displays a message whenever a record is deleted from the Student table.

```
CREATE OR REPLACE TRIGGER trg_student_delete  
AFTER DELETE ON Student  
BEGIN  
    DBMS_OUTPUT.PUT_LINE('Student Record Deleted');  
END;  
/
```



Question 2

Write a statement-level trigger that displays a message whenever a new teacher record is inserted into the Teacher table.

CREATE OR REPLACE TRIGGER trg_teacher_insert

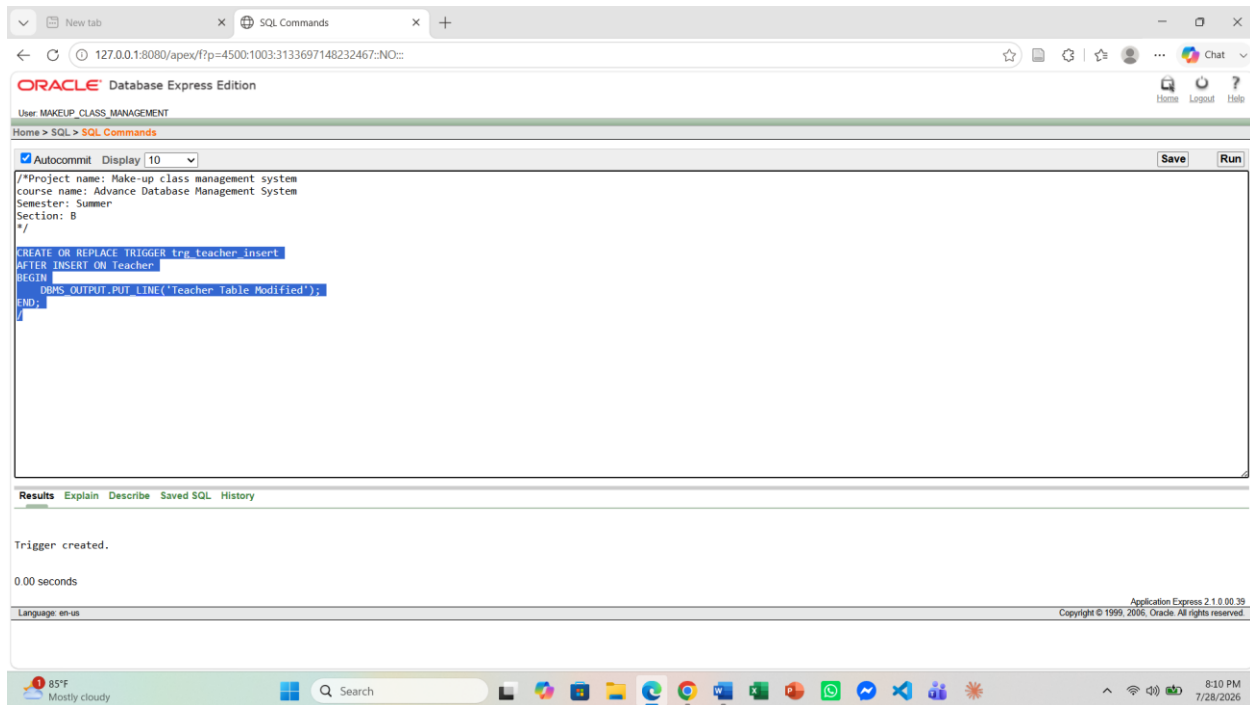
AFTER INSERT ON Teacher

BEGIN

 DBMS_OUTPUT.PUT_LINE('Teacher Table Modified');

END;

/



8. Package

Question 1

Write a PL/SQL package named student_pkg that contains a procedure to display a student's name using Student ID.

```
CREATE OR REPLACE PACKAGE student_pkg IS
```

```
    PROCEDURE show_student(p_sid NUMBER);
```

```
END;
```

```
/
```

```
CREATE OR REPLACE PACKAGE BODY student_pkg IS
```

```
    PROCEDURE show_student(p_sid NUMBER)
```

```
    IS
```

```
        v_name Student.s_name%TYPE;
```

```
BEGIN
```

```
    SELECT s_name
```

```
    INTO v_name
```

```
    FROM Student
```

```
    WHERE s_id=p_sid;
```

```

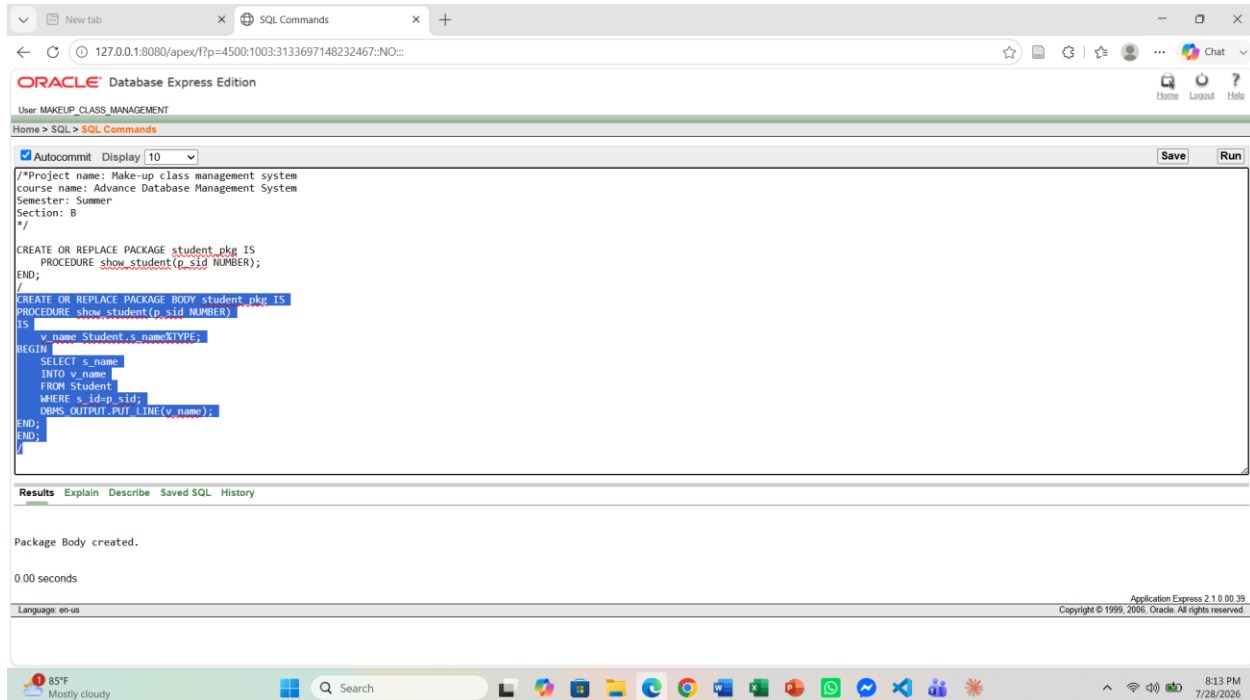
        DBMS_OUTPUT.PUT_LINE(v_name);

END;

END;

/

```



Question 2

Write a PL/SQL package named teacher_pkg that contains a function to return the total number of teachers.

```

CREATE OR REPLACE PACKAGE teacher_pkg IS

    FUNCTION total_teacher RETURN NUMBER;

END;

/

CREATE OR REPLACE PACKAGE BODY teacher_pkg IS

    FUNCTION total_teacher

    RETURN NUMBER

IS

    total NUMBER;

BEGIN

```

```

SELECT COUNT(*)

INTO total

FROM Teacher;

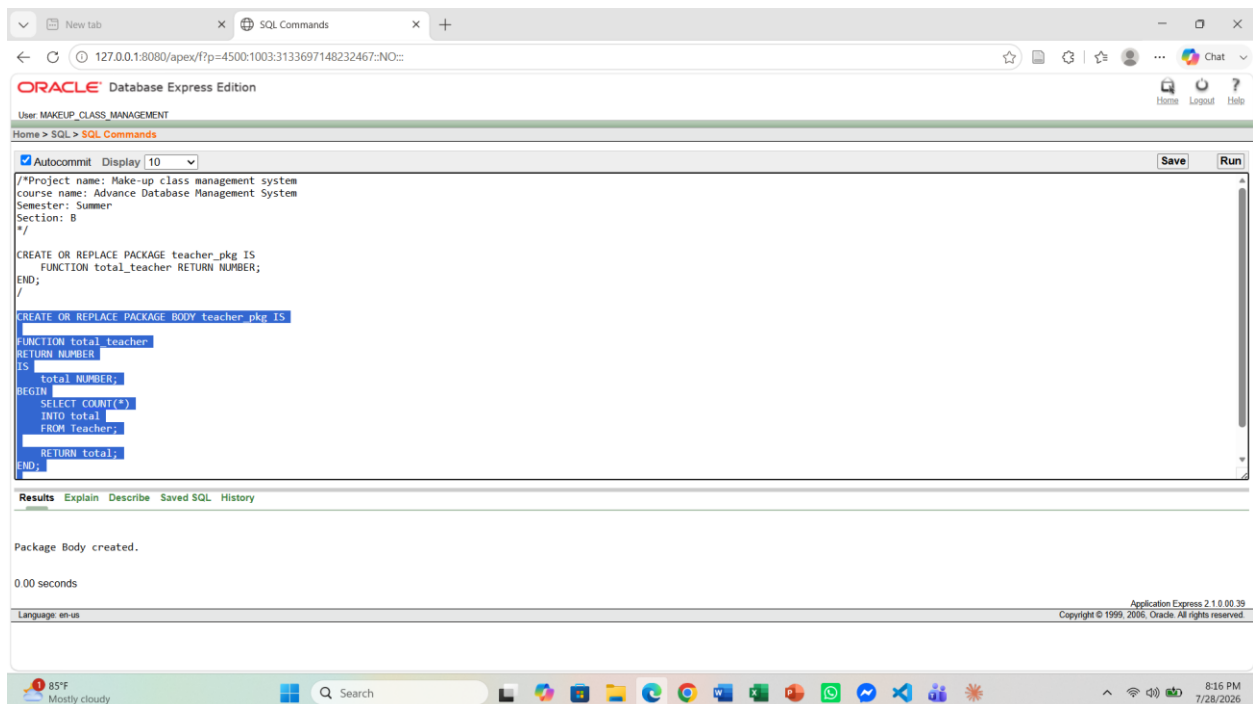
RETURN total;

END;

END;

/

```



Conclusion

The **Make-up Class Management System** project successfully demonstrates the design and implementation of a relational database using Oracle Database. The system provides an efficient way to manage departments, teachers, students, classrooms, make-up class requests, and notifications. Through proper database normalization and the use of SQL and PL/SQL features such as stored procedures, functions, cursors, triggers, and packages, the project ensures data integrity, minimizes redundancy, and improves database performance. Overall, the project simplifies the management of make-up class information and provides a reliable platform for maintaining accurate academic records.

Future Work

The current system can be further improved by adding several advanced features, including:

- Developing a user-friendly web or mobile interface for students and teachers.
- Implementing secure login and role-based access for administrators, teachers, and students.
- Adding automatic email or SMS notifications for make-up class schedules and approval status.
- Generating reports and analytics for attendance, classroom utilization, and request history.
- Integrating QR code or biometric attendance for make-up classes

These improvements will make the system more efficient, secure, scalable, and suitable for real-world academic environments.